WATERWISE: A RULE-BASED ADAPTIVE DIFFICULTY GAME FOR WATER REUSE EDUCATION AMONG FILIPINO ELEMENTARY STUDENTS

An Undergraduate Thesis Outline
Submitted to the Faculty of
College of Engineering and Information Technology
Cavite State University
Indang, Cavite

In partial fulfilment
of the requirements for the degree of
Bachelor of Science in Computer Science

RALPH LAURENCE J. CRUTO
RAEVEN M. ESPINELI
ELRONI F. QUIÑONES
February 2026

Table of Contents

INTRODUCTION..... 1
Statement of the Problem..... 2
Objectives of the Study..... 4
Hypothesis..... 4
Significance of the Study..... 5
General Use..... 5
End Users (Students)..... 6
Proponents..... 6
Scope and Limitations..... 7
Definition of Terms..... 9
REVIEW OF RELATED LITERATURE..... 12
Energy-Efficient Algorithms vs. Deep Learning on Mobile..... 12
Thermal Constraints and Algorithmic Efficiency on Edge Devices..... 13
Mobile Ad-Hoc Networks (MANETs) and Synchronization..... 13
Algorithmic Efficiency and Memory Management..... 13
Peer-to-Peer Architecture for Latency Reduction..... 14
Addressing Android Ecosystem Fragmentation..... 14
Portability and Cross-Architecture Optimization..... 15
The Application Domain: Water Conservation and Gamification..... 15
Educational Game-Based Learning Focused on Values..... 16
Rolling Window Rule-Based Adaptive Difficulty Algorithms..... 16
Theoretical Foundations of Conflict-Free Replicated Data Types (CRDTs).. 17
G-Counter Algorithm and Distributed Counting Semantics..... 17
Recent Advances in CRDT Implementation for Real-Time Distributed Systems..... 17
State-Based and Operation-Based CRDT Implementation in Educational Gaming..... 18
Implementation Challenges and Practical Considerations..... 18
Peer-to-Peer Architectures for Multiplayer Gaming..... 18
Distributed State Synchronization Mechanisms for Collaborative Gameplay..... 18
Multiplayer Collaborative Learning and Educational Game Design..... 19
Multiplayer Collaborative Gaming and Educational Benefits..... 19
CONTRIBUTION OF RRS TO THE PROPOSED SYSTEM..... 20
Table 1. Table of comparison of related studies..... 20
METHODOLOGY..... 22
Research Design..... 22
Target Demographic..... 22
Research Design and Approach..... 23
Work Plan..... 25
Pre-Sprint - Baseline Research and Stakeholder Consultation..... 25
Requirements and Architecture with Research Artifact Integration.... 26

Core Infrastructure Development....26
Mini-Game Development Batch 1 Addressing Survey-Identified Knowledge Gaps.... 27
Adaptive Difficulty Algorithm with Theoretical Validation.... 27
Mini-Game Development Batch 2 and Localization with Survey Language Validation.... 28
G-Counter CRDT Multiplayer Synchronization with Distributed Systems Validation....28
Integration, Refinement, and Post-Implementation Research Documentation....29
Integration Pattern Research Artifacts as Prioritized Backlog Items.. 29
Materials.... 30
Software Materials.... 30
Hardware Materials....30
Proposed Algorithm/Approach....31
Rule-Based Rolling-Window Adaptive Difficulty Algorithm.... 31
Circular Buffer Memory Allocation for Rule-Based Rolling Window....33
Mathematical Formulation.... 34
Rule-Based Decision Tree Logic.... 36
G-Counter CRDT Multiplayer Synchronization Algorithm....41
Mathematical Formulation.... 42
System Architecture....49
Conceptual Framework....53
Use Case Diagram.... 54
Implementation.... 55
Autoload Singleton Registration.... 55
Algorithm Integration Sequence.... 56
Performance Evaluation.... 58
Algorithmic Complexity Analysis Adaptive....61
Ethical Considerations....62
Voluntary Participation and Right to Withdraw....62
Privacy and Data Security.... 62
REFERENCES.... 63
APPENDICES.... 72
Appendix 1....73
Appendix 2....81
Appendix 3....89

WATERWISE: A RULE-BASED ADAPTIVE DIFFICULTY GAME FOR WATER REUSE EDUCATION AMONG FILIPINO ELEMENTARY STUDENTS

RALPH LAURENCE J. CRUTO
RAEVEN M. ESPINELI
ELRONI F. QUIÑONES

An undergraduate thesis outline submitted to the faculty of the Department of Information Technology, College of Engineering and Information Technology, Cavite State University, Indang, Cavite in partial fulfillment for the degree of Bachelor of Science in Computer Science with Contribution No. _____. Prepared under the supervision of Ms. Julie Ann C. Lontoc.

INTRODUCTION

Water scarcity is no longer just a distant environmental warning; it is a critical global reality. The United Nations Sustainable Development Goals (SDG 6) emphasize that sustainable water management is urgent, and education is the first line of defense. To bring this message to the youth, gamification has emerged as a powerful tool, transforming abstract environmental concepts into interactive experiences that students can actually understand and retain (Gómez Niño et al., 2024).

In the Philippines, this challenge is even more personal. Climate change and aging infrastructure have created a "water stress" scenario that demands immediate educational intervention (National Economic and Development Authority, 2023). However, bringing digital learning to Philippine classrooms faces a harsh technological reality: the "Digital Divide." Research confirms that the vast majority of students in developing regions do not own high-end smartphones; they rely on legacy Android devices with limited battery life, weak processors, and passive cooling systems (Rua & Saraiva, 2023).

This creates a significant engineering gap. While there are many educational apps available, most are not built for the hardware these students actually use.

High-fidelity games often push these older phones too hard, causing the CPU to overheat and trigger thermal throttling—a protective mechanism that slows down the device to prevent damage (PulseGeek, 2025). When this happens, the game lags, becomes unresponsive, and the educational value is lost. Furthermore, a preliminary requirements analysis conducted by the researchers at Indang Christian Academy (2025) highlighted that these students often lack reliable internet access, meaning the game must work perfectly offline. Yet standard synchronization methods fail when network connections are unstable or drop packets (Guidec et al., 2022).

Therefore, this study shifts the focus from content creation to technical optimization. The development of WaterWise serves as a testbed to validate specific engineering solutions: a lightweight Rule-Based Algorithm to prevent overheating, and an Integer-Only CRDT to ensure multiplayer data stays consistent without the internet. The primary objective is to prove that by optimizing computational efficiency and network protocols, we can deliver high-performance software within the strict hardware constraints of the Filipino classroom.

Statement of the Problem

The deployment of gamified educational applications in developing regions is critically hindered by the hardware and network constraints of the target environment. A primary technical challenge is the prohibitive energy cost of standard Dynamic Difficulty Adjustment (DDA) systems which frequently utilize deep learning models ill-suited for battery-dependent devices. Jung et al. (2025) report that the local processing required for these complex algorithms can increase energy consumption by up to 94% and significantly reduce gameplay duration. Consequently there is a lack of lightweight and heuristic-based alternatives specifically engineered to

minimize this energy overhead on the low-specification hardware found in Philippine schools.

This high energy consumption directly exacerbates the issue of thermal throttling on legacy Android devices particularly those running on Cortex-A53 architectures. Benoit-Cattin et al. (2020) demonstrated that sustained CPU load from heavy inference tasks triggers protective thermal mechanisms within seconds which forces the operating system to drastically lower clock speeds to prevent hardware damage. This thermal generation poses both a technical and ergonomic threat. Kang et al. (2019) found that high-load processing can raise device temperatures to uncomfortable levels with 75% of users reporting that such heat disrupts their interaction flow.

The impact of this throttling is particularly critical for the target "micro-game" genre where reaction windows are often under 500ms. Sahin et al. (2015) quantified that aggressive thermal management can increase "frame rate defects" or lag spikes by up to 146%. In an educational context this latency results in "unfair player failure" where a student fails not due to a lack of knowledge but because the device could not render the frame in time. Current educational software often prioritizes feature density over algorithmic efficiency and fails to implement the (O(1)) complexity standards necessary to maintain performance stability under these strict thermal limits.

Furthermore the reliance on unstable mobile hotspots for multiplayer interaction introduces significant data consistency challenges. Guidec et al. (2022) established that standard synchronization protocols and JSON-based Conflict-Free Replicated Data Types (CRDTs) suffer from metadata bloat which renders them ineffective in opportunistic networks characterized by high packet loss. This creates a specific need for an integer-only synchronization mechanism capable of ensuring state convergence without the bandwidth overhead inherent in existing libraries.

Objectives of the Study

The general objective of this study is to develop "WaterWise," a rule-based adaptive difficulty game with offline multiplayer capabilities. Specifically, the study aims to:

1. To develop the Rule-Based Algorithm with Rolling Window Technique and evaluate its Performance Efficiency (ISO 25010) by measuring Clock Speed Stability and Frame Rate consistency on low-end Android devices (Cortex-A53) during continuous 30-minute stress tests.
2. To quantify the Energy Efficiency of the system by benchmarking battery discharge rates (mAh) against a baseline Deep Learning model, validating the efficiency gains projected by Jung et al. (2025).
3. To evaluate the Reliability (ISO 25010) of the G-Counter CRDT synchronization mechanism under simulated high-latency conditions, specifically testing convergence times at 10% to 90% packet loss rates as characterized by Guidec et al. (2022) for opportunistic mobile networks.

Hypothesis

Based on the identified hardware constraints of Philippine educational environments and the proposed WaterWise system architecture, this study hypothesizes that the Rule-Based Rolling-Window Adaptive Difficulty Algorithm will achieve constant-time (O(1)) computational complexity, thereby maintaining Clock Speed Stability and preventing thermal throttling (keeping CPU temperatures <45°C) on low-specification Cortex-A53 Android devices. Concurrently, the Integer-Only G-Counter CRDT synchronization mechanism is hypothesized to achieve state convergence within 200ms and maintain 100% data consistency even under simulated 90% packet loss conditions, significantly outperforming standard

JSON-based serialization in bandwidth-constrained mobile ad-hoc networks. From a pedagogical perspective, the system is expected to facilitate sustained engagement and measurably reduce the 53% baseline misconception rate regarding water safety classification by keeping students within their Zone of Proximal Development. Finally, the complete software artifact will meet the ISO/IEC 25010 Software Product Quality standards, specifically excelling in Performance Efficiency (battery drain <10mAh per 5-minute session) and Reliability (serverless fault tolerance), demonstrating that deterministic heuristics are a viable, energy-efficient alternative to deep learning for edge-based educational gaming.

Significance of the Study

This research holds substantial significance for stakeholders in green computing, educational technology, and environmental advocacy by establishing a technical framework for deploying high-performance educational software on legacy hardware within resource-constrained environments.

General Use

WaterWise directly contributes to the field of Green Computing and Energy Efficiency by demonstrating that deterministic, rule-based heuristics can replace computationally expensive Deep Learning models for dynamic difficulty adjustment. By validating an algorithm that reduces CPU load and energy consumption, the study offers a replicable model for extending the battery life and operational longevity of mobile devices in developing regions, thereby indirectly reducing e-waste. Furthermore, the system supports Edge Gaming and serverless architectures, proving that multiplayer experiences can be delivered through peer-to-peer mobile ad-hoc networks (MANETs) without the need for costly centralized cloud infrastructure or persistent internet connectivity, which promotes technological equity in remote Philippine areas.

End Users (Students)

For students in the Philippines, specifically those impacted by the "Digital Divide," WaterWise provides a seamless educational experience that functions optimally on low-end, legacy Android devices without triggering thermal throttling or frame rate instability. By utilizing a lightweight Rule-Based Algorithm rather than heavy AI, the system ensures that learners with older hardware are not excluded from interactive learning due to lag or overheating. Educationally, the game addresses critical knowledge gaps identified in preliminary surveys specifically the 53% of students unable to distinguish safe water sources by allowing them to practice water reuse logic (e.g., SortWater mini-games) in a culturally responsive, bilingual (English-Filipino) environment.

Proponents

This study transitions the proponents from content creators to Algorithmic Engineers by shifting the focus from simple game design to technical optimization and complexity analysis. The proponents gain specialized expertise in implementing Conflict-Free Replicated Data Types (CRDTs), specifically the G-Counter algorithm, to solve complex distributed systems challenges like "metadata bloat" and packet loss in offline networks. Furthermore, the development process validates their ability to enforce strict (O(1)) time complexity standards and memory-safe architectural patterns (Fixed-Size Circular Buffers), equipping them with high-value skills in systems programming and performance profiling required for the software engineering industry.

Future Researchers

This research contributes to the academic body of knowledge by challenging the prevailing assumption that "more computation is better," offering empirical evidence that Deterministic Heuristics > Heavy AI for edge gaming applications. It

provides future researchers with a validated technical artifact the Integer-Only G-Counter CRDT which serves as a foundation for developing "Offline-First" collaborative applications capable of surviving 90% packet loss in opportunistic networks. Additionally, the study's rolling window approach offers a generalizable framework for real-time decision-making that researchers can adapt for other edge computing domains, such as IoT sensor processing or anomaly detection on battery-dependent devices.

Scope and Limitations

The proposed study focuses on the design, development, and algorithmic validation of WaterWise, a 2D cross-platform educational game specifically engineered for Filipino elementary students. The system is built using the Godot 4.5 Game Engine to ensure deployment viability on resource-constrained hardware, specifically targeting legacy Android devices (Cortex-A53 architecture, <2GB RAM) and standard Windows workstations found in Philippine public school laboratories. The primary scope encompasses the creation of 19 culturally responsive mini-games centered on water conservation and reuse, driven by a proprietary Rule-Based Adaptive Difficulty engine and an offline-first G-Counter CRDT multiplayer synchronization system. The study does not cover broad environmental science topics beyond water management, nor does it support iOS deployment, cloud-based data storage, or online matchmaking requiring active internet connectivity.

Presentation Module. The Presentation Module encompasses the graphical user interface (UI) and user experience (UX) components, including the Initial Screen, Main Menu, In-Game HUD, and Character Customization interfaces. This module is designed to process touch inputs for mobile devices and mouse/keyboard inputs for desktop environments. The visual scope is strictly limited to 2D sprite-based animations and lightweight UI elements to strictly enforce the 16ms

frame budget required for 60 FPS on low-end hardware; it will not support 3D rendering, dynamic lighting, or particle-heavy effects that trigger thermal throttling. Furthermore, while the interface supports real-time English and Filipino localization via a hash-map lookup system, it does not include support for other regional dialects or assistive technologies for visually impaired users beyond basic text scaling.

Business Logic Module. The Business Logic Module defines the core gameplay mechanics through a modular inheritance structure, utilizing a MiniGameBase abstract class to govern the logic of specific mini-games such as CatchTheRain, BucketBrigade, FixLeak, and SortWater. This module handles lightweight physics collisions (AABB/Circle), score calculation, and timer management using deterministic logic. The scope is confined to finite, arcade-style gameplay loops designed for short burst sessions (1-3 minutes); it excludes open-world exploration, complex rigid-body physics simulations, or infinite procedural generation to maintain constant-time ($O(1)$) computational complexity.

Service Module. The Service Module functions as the system's central intelligence, implementing the Rule-Based Rolling-Window Adaptive Difficulty Algorithm and the G-Counter Conflict-Free Replicated Data Type (CRDT) for network synchronization. The adaptive system calculates a Proficiency Index (**Φ**) based on a fixed-size circular buffer of the last five games, utilizing weighted moving averages and standard deviation for consistency penalties. A critical limitation is that this module explicitly excludes Deep Learning, Neural Networks, or AI-based inference to minimize battery drain and thermal throttling. Additionally, the NetworkManager is restricted to serverless, peer-to-peer UDP communication (Port 7777) over local ad-hoc networks (MANETs); it does not support wide-area network (WAN) connectivity, centralized cloud servers, or anti-cheat enforcement mechanisms.

Data Persistence Module. The Data Persistence Module manages the local storage and retrieval of application data, utilizing the Godot ConfigFile API for user

preferences (volume, language) and generating JSON exports for gameplay telemetry and session performance metrics. The system operates on a Local-First Architecture, ensuring that all progress and logs are saved directly to the device's internal storage. Consequently, the study limits data handling to offline contexts; it will not support cloud database integration (e.g., Firebase, AWS), cross-device save synchronization, or remote analytics dashboards. Security is limited to local file permissions and binary serialization, without advanced encryption standards for the exported JSON logs.

Definition of Terms

The following terms are defined as used within the context of this research

Adaptive Difficulty Algorithm. A computational system that automatically adjusts game challenge levels based on real-time analysis of player performance data, utilizing rolling-window methodology to assess recent results.

Conflict-Free Replicated Data Type (CRDT). A distributed data structure enabling strong eventual consistency across multiple independent nodes without centralized coordination, allowing peer-to-peer data synchronization through mathematically-proven commutative operations.

Flow State. The optimal engagement zone where challenge perfectly matches skill level, representing the sweet spot between anxiety and boredom where individuals realize peak performance and deep engagement.

G-Counter Algorithm. A grow-only counter CRDT implementation enabling conflict-free score synchronization across distributed multiplayer devices with O(1) time complexity for increment operations.

GDScript. Python-like programming language designed for ease of use in the Godot game engine, facilitating rapid development while maintaining code clarity and maintainability.

Godot 4.5. An open-source, cross-platform game development engine specifically selected for its flexibility, scene-based modular architecture, and suitability for educational game development.

Greywater. Wastewater generated from household activities such as laundry, dishwashing, and bathing that can be safely reused for non-potable purposes like toilet flushing and plant watering.

ISO/IEC 25010 Software Product Quality Model. An international standard for evaluating software quality across eight characteristics, functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability.

Mini-game. A short, focused game activity that presents a specific water conservation challenge within the larger educational game, designed to take 1-3 minutes to complete.

Proficiency Index (Φ). A unified metric combining accuracy and consistency performance that serves as the primary decision variable for adaptive difficulty adjustments, ranging from -0.2 to 1.0.

Rolling Window. A data analysis technique that maintains only the five most recent performance records while discarding older data that no longer reflects current ability, ISOoperating through FIFO (first-in, first-out) mechanisms.

Rule-Based System. A computational approach that applies predetermined logical conditions (if-then rules) to make decisions, offering transparency and explainability compared to complex machine learning models.

Water Reuse. The practice of using water more than once for the same or different purposes, such as collecting rainwater for cleaning or using vegetable washing water for plants.

Zone of Proximal Development (ZPD). The space where tasks are beyond current abilities but attainable with the right level of challenge, where the brain must stretch, adapt, and develop new strategies without hitting the frustration wall.

REVIEW OF RELATED LITERATURE

This chapter presents the key discussion, concepts, and sources that will form both the information and theoretical foundations of this study. It will include the supporting literature and related works that help strengthen the direction of the research. The reviewed materials in this chapter serve as guides in shaping the research methods, as well as in outlining the objectives and overall flow of the study.

Energy-Efficient Algorithms vs. Deep Learning on Mobile

Green computing minimizes energy consumption which is critical for mobile gaming where high CPU usage depletes batteries. While Dynamic Difficulty Adjustment maintains engagement existing solutions rely on expensive AI models that impose heavy loads unsuitable for low-end devices (Marmat & Thankachan 2025). Kim et al. (2021) found that AI-driven thermal management introduces its own energy overhead while PulseGeek (2025) notes that algorithmic complexity causes thermal spikes. This study addresses that gap by replacing deep learning with a deterministic Rule-Based Rolling Window Algorithm effectively eliminating the "AI tax" on battery life.

The Impact of Game Loops on Thermal Throttling

Mobile games rely on a continuous Game Loop that keeps processors active. Rua and Saraiva (2023) found that high frame rates are the primary battery drain while complex logic causes thermal runaway exceeding 45°C. PulseGeek (2025) notes this triggers throttling and stuttering. Consequently Energy-Aware loops using O(1) logic are essential to allow the CPU to sleep between frames.

Thermal Constraints and Algorithmic Efficiency on Edge Devices

Recent literature emphasizes the link between algorithmic complexity and mobile thermal limits. Benoit-Cattin et al. (2020) revealed that Deep Learning triggers thermal throttling on uncooled devices causing immediate performance degradation. Sahin et al. (2015) showed that this throttling increases lag spikes by 146% disrupting the user experience. Furthermore Kang et al. (2019) found that high-load tasks raise device temperatures over 50°C causing physical discomfort for 75% of users.

Mobile Ad-Hoc Networks (MANETs) and Synchronization

Mobile Ad-Hoc Networks in P2P gaming face challenges like variable latency and packet loss. Dantas and Baquero (2025) showed that CRDTs reduce latency by 75% compared to servers but suffer from metadata bloat that clogs bandwidth. Saquib et al. (2022) argued that standard CRDTs prioritize theoretical consistency over practical throughput in unstable networks. Sypytkowski (2021) notes that even optimized versions carry heavy overhead for causality tracking. This validates the need for Integer-Only Payload Optimization which strips metadata to ensure convergence even during the 90% packet loss scenarios common in Philippine educational settings.

Algorithmic Efficiency and Memory Management

The choice of data structures is critical for maintaining performance on devices with limited RAM. While dynamic arrays are standard in high-level game development they incur significant overhead during memory reallocation. Upadhya et al. (2025) demonstrated that sliding window models significantly outperform other stream mining techniques in memory efficiency by processing data streams in a single pass without the need for backtracking or dynamic resizing. Furthermore

technical documentation on real-time systems emphasizes that circular buffers enable lock-free concurrency and constant-time memory access which is essential for preventing garbage collection spikes that cause frame drops in interactive applications (AlgoCademy, 2025).

Peer-to-Peer Architecture for Latency Reduction

For multiplayer functionality in areas with poor internet connectivity a Peer-to-Peer or P2P architecture offers distinct advantages over traditional client-server models. Yahyavi and Kemme (2025) in their survey of Massively Multiplayer Online Games found that P2P architectures effectively reduce latency by removing the server hop and allowing clients to communicate directly. However they noted that state consistency remains a challenge in these decentralized systems. Recent research confirms that Conflict-Free Replicated Data Types or CRDTs allow for eventual consistency without the heavy bandwidth overhead of locking mechanisms which makes them ideal for the high-packet-loss environments typical of mobile hotspots (Guidec et al., 2022).

Addressing Android Ecosystem Fragmentation

The diversity of the Android ecosystem poses a unique challenge known as fragmentation where an application must run on thousands of different device configurations. Trio Content Team (2025) reports that as of late 2025 fragmentation remains a critical issue with only 4.5% of devices running the latest Android version while a vast majority rely on older and less secure iterations. This diversity forces developers to optimize for the lowest common denominator to avoid crashes and performance inconsistencies. Alphabin (2025) further highlights that fragmentation increases the risk of device-specific defects where an app works on one phone but fails on another due to differing screen sizes or chipsets.

Algorithmic Foundations of Rolling Window Heuristics

Selecting the right data structure is critical for maintaining O(1) time complexity. Algocademy (2025) confirms that Circular Buffers provide constant-time operations ideal for real-time systems. Mees (2018) validates that Weighted Rolling Averages estimate learner ability more accurately than raw scores. While Lopes et al. (2025) note a lack of lightweight frameworks this research bridges that gap using Fixed-Size Circular Buffers. This approach prevents garbage collection spikes and frame drops on Android devices.

Portability and Cross-Architecture Optimization

The Android ecosystem suffers from extreme hardware fragmentation between ARM and x86 architectures causing significant performance variance (Intelivita, 2025). Godot 4's Vulkan renderer struggles on low-end devices despite abstraction. Therefore efficient mobile development requires optimizing underlying logic using lightweight algorithms like Rolling Window and Integer-Only CRDT to reduce workload. This ensures playability on diverse low-cost hardware found in Philippine schools by minimizing computational overhead. (Quinnox, 2025)

The Application Domain: Water Conservation and Gamification

While the primary focus of this study is algorithmic optimization, the application domain of water conservation was selected due to urgent environmental necessities. The Philippines faces intensifying water stress projected to worsen from climate change, creating a critical need for effective educational intervention. Shohieb et al. (2025) validated that adaptive serious games significantly improve learning outcomes by keeping learners in their "Zone of Proximal Development," a finding supported by Chiotaki et al. (2023), who noted that gamification enhances retention compared to traditional methods.

To ensure this pedagogical validity applies locally, the game content was derived from a diagnostic survey of 100 elementary students and expert consultations. Baseline research revealed critical misconceptions, notably that 53% of students could not distinguish safe from unsafe water sources, necessitating specific mini-games like SortWater for classification logic. Furthermore, 79% of the demographic preferred Filipino learning materials. Consequently, this study implements the adaptive benefits identified by Shohieb et al. within a bilingual framework, ensuring the technical artifact is both culturally responsive and pedagogically grounded.

Educational Game-Based Learning Focused on Values

The "Ugaling Pinoy" mobile learning game demonstrates that culturally contextualized educational games effectively foster values education and behavioral change among Filipino elementary students through interactive gameplay featuring quizzes, fill-in-the-blank challenges, and scenario-based activities that facilitate both knowledge acquisition and positive attitudinal shifts (Escultor, Moises, & Sales, 2023).

Collaborative Environmental Education through Games

Environmental education games promote ecological behaviors by combining hands-on learning with immersive environments (Tsai et al. 2025). Integrating community activities with games fosters lasting habits (Abrencia 2025) while collaborative gameplay strengthens environmental reasoning and cognitive outcomes (Janakiraman et al. 2021).

Rolling Window Rule-Based Adaptive Difficulty Algorithms.

The rolling window technique keeps learners in their Zone of Proximal Development outperforming static difficulty (Shohieb et al. 2025). Rule-based systems offer transparency and stability valuable for education. However Lopes et al. (2025) note a gap in applying this to 2D games on resource-constrained devices

while reviews suggest using collective metrics to maintain motivation (Chiotaki et al. 2023).

Theoretical Foundations of Conflict-Free Replicated Data Types (CRDTs).

Conflict-free Replicated Data Types enable seamless replication without coordination bottlenecks ensuring deterministic convergence through strong eventual consistency (Preguiça et al. 2018). State-based approaches using delta transmission are particularly useful for Philippine schools as they reduce bandwidth overhead while maintaining consistency (Shapiro et al. 2011 Almeida et al. 2018). This robustness is proven in major systems like League of Legends.

G-Counter Algorithm and Distributed Counting Semantics.

The G Counter is a state based CRDT using a local array where replicas increment only their own entry. Merging takes the element wise maximum and sums entries for a global count guaranteeing convergence. This offers low latency and partition tolerance for resource constrained educational systems without heavy infrastructure (Preguiça et al 2018). Limitations include increment only support and array growth with replicas making it best for stable groups like school sessions while PN Counters better suit dynamic membership or decrements (Preguiça et al 2018).

Recent Advances in CRDT Implementation for Real-Time Distributed Systems.

Contemporary research focuses on optimizing CRDTs for practical use. Dantas and Baquero (2025) demonstrated that CRDTs support responsive real time VR by treating latency as a data consistency issue rather than just a network problem. Their work on operation based CRDTs using vector clocks and state based CRDTs with delta transmission offers valuable insights for multiplayer educational games. Additionally industrial systems like Yjs often outperform academic prototypes by utilizing specialized data structures and realistic testing methods (Preguiça et al., 2018).

State-Based and Operation-Based CRDT Implementation in Educational Gaming.

State-based CRDTs reconcile differences by comparing complete state vectors ensuring results remain independent of message order but requiring higher bandwidth (Sypytkowski 2021). Operation-based CRDTs transmit only incremental operations to reduce storage but shift complexity to the replication protocol to ensure causal ordering (Shapiro et al. 2011).

Implementation Challenges and Practical Considerations.

Dantas and Baquero (2025) documented challenges in distributed systems and network fairness within BrickSync yet performance analysis showed latency averaging 50 milliseconds a 75% reduction compared to remote servers. G-Counter CRDTs track player proficiency by counting successes without synchronizing complex values (Shapiro et al. 2011). However ensuring consistent calculations requires carefully designed algorithms particularly when one player's level affects shared parameters (Dantas & Baquero 2025).

Peer-to-Peer Architectures for Multiplayer Gaming.

Peer-to-peer architectures distribute tasks to improve scalability but require latency management (Jiang 2012). Research shows client-server models suit competitive games while P2P fits cooperative educational apps. Modern hybrid systems balance server load and individual tracking by combining server authority with P2P synchronization (Viitanen 2025).

Distributed State Synchronization Mechanisms for Collaborative Gameplay.

Multiplayer games require consistent state maintenance across devices using models like server authoritative or lockstep depending on the genre (Kroupp 2024). However cooperative educational games can tolerate temporary divergence making CRDT-based synchronization an ideal solution. Because CRDT updates converge deterministically without centralized servers they effectively address the infrastructure constraints found in Philippine schools (Dantas & Baquero 2025).

Multiplayer Collaborative Learning and Educational Game Design.

Collaborative learning research proves peer interaction drives deeper understanding than isolated study. Multiplayer games leverage this by improving self management through shared objectives while MMORPG systems support real time collaboration (Velásquez et al. 2024 Ye et al. 2025). Furthermore integrating adaptive difficulty tailors challenges to individual skills enabling students of varying abilities to collaborate effectively.

Multiplayer Collaborative Gaming and Educational Benefits.

Multiplayer educational games enhance learning by fostering social collaboration and critical thinking. Studies confirm that peer interaction drives better attitudinal outcomes than traditional instruction (Hatta et al. 2024 Janakiraman et al. 2021). Furthermore integrating cooperative elements with meaningful progression systems significantly increases student motivation and completion rates (Canella et al. 2025).

Synthesis

The review of literature confirms two distinct technical problems in edge computing, specifically the energy inefficiency of deep learning models and the bandwidth inefficiency of standard synchronization protocols in MANETs. A clear gap exists regarding the replacement of AI with deterministic heuristics to explicitly solve thermal throttling in offline educational games. Furthermore, there is a lack of simplified, integer-only CRDT implementations designed specifically for high-loss opportunistic networks. This research bridges these gaps by engineering the WaterWise system, which replaces AI inference with an O(1) Rule-Based Adaptive Engine to address energy deficits and replaces JSON CRDTs with a Binary G-Counter to resolve bandwidth constraints.

CONTRIBUTION OF RRS TO THE PROPOSED SYSTEM

Table 1. Table of comparison of related studies

Feature/Domain	Standard Approach (The Research Gap)	WaterWise Optimization (The Solution)	Impact on Low-End Mobile (Why it Matters)	Supporting Literature (Citations)
Adaptive Difficulty	Deep Learning / AIReliance on heavy inference models increases energy consumption by up to 94% and triggers thermal throttling.	Rule-Based Rolling WindowReplaces heavy inference with deterministic heuristics using Weighted Moving Averages (WMA).	Prevents Thermal ThrottlingEliminates the "AI tax" on battery life, keeping device temperatures <45◦C.	Jung et al. (2025): AI inference increases energy by 94%. Marmat & Thankachan (2025): AI pipelines are too heavy for edge devices.
Algorithmic Complexity	Variable Complexity (O(N2))Complex logic (like matrix multiplication) keeps the CPU active, preventing sleep states.	Constant Time (O(1))Utilizes pointer arithmetic to ensure logic execution time remains negligible regardless of session length.	Clock Speed StabilityEnsures the Game Loop stays within the 16ms budget (60 FPS), avoiding "frame rate defects."	Kim et al. (2021): Unoptimized logic triggers thermal spikes. PulseGeek (2025): Algorithmic complexity causes thermal runaway.
Memory Management	Dynamic ArraysFrequent memory reallocation for growing data streams causes Garbage Collection (GC) spikes.	Fixed-Size Circular BufferPre-allocates memory blocks to handle data streams in a single pass without resizing.	Memory Safety on <2GB RAMPrevents heap fragmentation and performance stutters on legacy Android devices.	Algocademy (2025): Circular buffers ensure O(1) memory safety. Upadhya et al. (2025): Sliding windows outperform stream mining in memory efficiency.

Network Synchronization	JSON-Based CRDTsStandard libraries prioritize theoretical consistency but suffer from "metadata bloat" that clogs bandwidth.	Integer-Only G-CounterStrips all metadata, transmitting only a bit-packed binary payload (approx. 4 bytes).	90% Packet Loss SurvivalEnsures state convergence in opportunistic ad-hoc networks where bandwidth is scarce.	Guidec et al. (2022): Standard CRDTs fail in high-loss opportunistic networks due to bloat. Sypytkowski (2021): Metadata overhead is a critical bottleneck.
System Architecture	Client-Server / CloudRelies on centralized coordination, adding latency (server hops) and requiring internet.	Serverless P2P (Offline-First)Facilitates direct device-to-device synchronization via UDP Port 7777 without external dependencies.	Reduced Latency (<16ms)Reduces latency by ~75% by removing the server hop; works in remote areas.	Dantas & Baquero (2025): P2P CRDTs reduce latency by 75% compared to servers. Yahyavi & Kemme (2013): P2P architectures remove server bottlenecks.
Ergonomics	High-Load ProcessingSustained CPU usage raises surface temperatures >50◦C, causing physical discomfort.	Energy-Aware LogicOptimized O(1) algorithms allow the CPU to enter low-power states between frames.	Extended PlayabilityMaintains comfortable device temperatures, ensuring students remain engaged.	Kang et al. (2019): High-load tasks raise temps >50◦C, disrupting 75% of users. Benoit-Cattin et al. (2020): Heavy inference triggers thermal throttling.

METHODOLOGY

This research employs a Scrum-Agile development methodology to design, construct and rigorously validate the WaterWise system utilizing a seven-sprint iterative lifecycle to isolate and solve specific computational challenges found in edge computing environments. The methodology is fundamentally experimental, focusing on the construction of two novel algorithmic artifacts: a Rule-Based Rolling-Window Adaptive Difficulty engine and a G-Counter Conflict-Free Replicated Data Type (CRDT) for synchronization. Unlike traditional waterfall approaches, the Agile framework allows for the continuous profiling of algorithmic complexity against the hardware constraints of low-end Android devices enabling the research team to rigorously enforce an O(1) time complexity budget throughout the development process. By adopting this approach the study systematically isolates technical variables specifically thermal throttling and network instability to demonstrate that deterministic heuristics can outperform deep learning models in terms of energy efficiency and frame rate stability addressing the critical performance gaps identified by Rua and Saraiva (2023) regarding mobile application energy consumption.

Research Design

Target Demographic

While the application domain involves education for Filipino elementary students, the technical validation of the system targets the computational constraints of the deployment environment rather than pedagogical outcomes alone. The elementary student demographic is utilized in this study primarily as a source of stochastic input data. Their interaction with the system generates the high-variance telemetry—specifically erratic reaction times and frequent input errors—required to stress-test the adaptive difficulty algorithm's ability to filter noise and detect proficiency shifts under real-world conditions. By treating these users as stochastic

data generators, the research can rigorously evaluate the algorithm's ability to filter outlier data and detect genuine proficiency shifts without the computational overhead of complex neural networks. Furthermore, the deployment environment targets low-end Android devices with limited RAM (<2GB) and passive cooling, creating a hostile hardware environment that validates the algorithm's efficiency claims better than high-end simulation would.

Research Design and Approach

The study utilizes a mixed-methods experimental design explicitly centered on software engineering and algorithmic performance analysis. The primary mode of inquiry is Constructive Research, where the core contribution is the technical artifact itself—specifically, the optimized algorithms embedded within the application. This design operationalizes the software system not merely as an educational tool, but as a technical testbed to validate two specific engineering hypotheses. First, it posits that a deterministic rule-based heuristic utilizing a rolling window technique can achieve difficulty adjustment responsiveness superior to deep learning models on legacy hardware, specifically by avoiding the thermal throttling triggers described by Kim et al. (2021). Second, it hypothesizes that an integer-only G-Counter CRDT payload can maintain state consistency in high-loss ad-hoc networks where standard JSON-based synchronization fails due to metadata bloat, as characterized by Rinberg et al. (2022). This ensures that the research focus remains on quantifying the computational efficiency, thermal stability, and network robustness of the proposed algorithms rather than solely evaluating pedagogical content.

To validate these hypotheses, the research design operationalizes theoretical frameworks into functional code through two distinct but interconnected phases of algorithmic engineering. First, the study focuses on algorithmic construction, designing the Rule-Based Rolling Window algorithm to instantiate Flow Theory and

the G-Counter CRDT to facilitate collaborative equity without server dependencies. Second, the research progresses to system prototyping, producing a functional cross-platform game developed in Godot 4.5 that serves as the telemetry instrument for these algorithms. This distinction separates the study from purely theoretical work by adhering to rigorous practical constraints, such as the hardware limitations of Philippine school computer laboratories, ensuring the resulting artifact is deployable in real-world edge computing environments.

The quantitative engagement analysis involves the systematic collection of behavioral and performance metrics captured during gameplay through the system's built-in data logging mechanisms. These metrics include session duration, task completion rates relative to CPU usage, and difficulty progression patterns, which are automatically logged by the Data Persistence Layer through JSON export functionality. This telemetry enables a systematic quantitative analysis of algorithmic performance, correlating logical game states with hardware stress indicators to demonstrate that the optimized code maintains functionality where traditional approaches would fail.

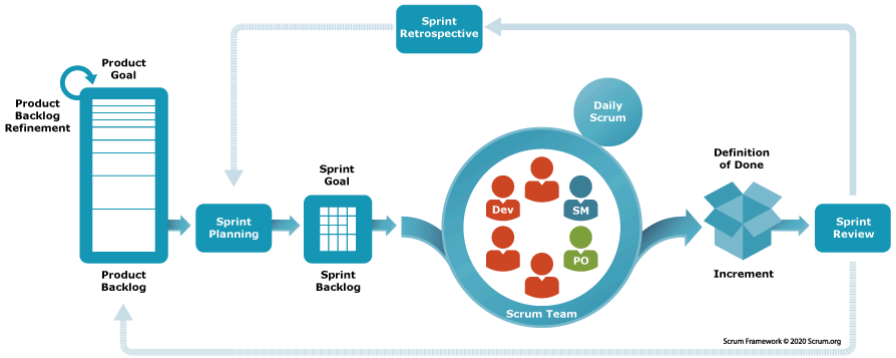


Figure 1. The Scrum framework (Scrum.org, n.d.)

Work Plan

Pre-Sprint - Baseline Research and Stakeholder Consultation

Prior to the commencement of the first sprint, the research team focused on defining the boundary conditions for the algorithmic logic and establishing the computational budget. This phase involved translating qualitative survey data regarding user misconceptions into boolean logic gates for the classification algorithms, ensuring the system could programmatically distinguish between specific error types. Simultaneously, a hardware profiling audit was conducted to identify the thermal throttling thresholds of target devices, explicitly defining the maximum allowable CPU cycles per frame. This pre-sprint analysis established the hard requirement that all subsequent algorithmic implementations must utilize static memory allocation to avoid garbage collection spikes that would violate the 16ms frame budget, a constraint emphasized by the Godot Community (2024) for maintaining responsiveness on legacy hardware.

Integration Pattern - Research Artifacts as Prioritized Backlog Items

Both survey and interview activities produced quantitative datasets and qualitative insights that the Product Owner systematically translated into prioritized Product Backlog items during Sprint 1 planning, ensuring development remained tightly coupled to empirical findings rather than technological feasibility alone. Survey results showing that 47% of students rarely think about water scarcity led to high-priority items such as "Create ScarcityAwareness mini-games emphasizing water crisis urgency" and "Integrate authentic water shortage scenarios reflecting Philippine context," while the 53% misconception about unsafe water types produced the "Develop SortWater mini-game addressing water quality classification" item, with content guided by expert-documented household misconceptions, such as equating visual clarity or lack of odor with safety. Interview findings about infrastructure

barriers, specifically small lot sizes, shared drainage, and informal plumbing, generated the backlog item "Design realistic household scenarios reflecting actual Filipino water management constraints," which was implemented across Sprints 3 and 5. Furthermore, survey validation that 79% of the demographic prefer Filipino for understanding educational materials yielded the high-priority "Implement English-Filipino localization system," emphasized in Sprint 5, while expert context on seasonal droughts and flood contamination informed the backlog item "Integrate environmental factors and climate challenges into game narratives," ensuring educational content reflected the real-world complexity of household water management in Philippine settings.

Requirements and Architecture with Research Artifact Integration

The first sprint focused on translating the computational constraints into a rigorous System Architecture and defining the core data structures. The team designed a modular four-layer architecture that strictly decouples the Service Layer where the core algorithms reside from the Presentation Layer. This separation is critical for the thesis as it allows for the independent profiling of algorithmic latency without interference from rendering overhead. During this phase the data structures for the Rolling Window algorithm were formalized. The team opted for a fixed-size Circular Buffer rather than a dynamic list to enforce O(1) space complexity. This architectural decision was documented as a primary method for preventing heap fragmentation on low-RAM devices directly addressing the hardware constraints identified in the pre-sprint phase and ensuring stable memory usage over extended runtime.

Core Infrastructure Development

Sprint 2 established the foundational telemetry pipeline required to feed the adaptive algorithms. The focus shifted to engineering the "Game Manager" singleton,

which functions as the central state machine. The research team implemented a high-frequency data logging system capable of capturing user input events with millisecond precision. This infrastructure is not merely a game feature but the data acquisition layer for the research; it standardizes raw inputs (taps, drags) into normalized vector formats (Accuracy, ReactionTime) that the adaptive engine can process. Technical validation during this sprint ensured that the logging mechanism itself introduced negligible overhead, preserving the integrity of the performance data required for the subsequent algorithmic validation.

Mini-Game Development Batch 1 Addressing Survey-Identified Knowledge Gaps

In this phase, the research team integrated the first set of interactive modules, functioning as data generators for the algorithm. The primary technical task was the implementation of the physics engine and collision detection logic that underpins the telemetry. By deploying modules such as "SortWater" and "CatchTheRain," the team began generating the specific input patterns—rapid classification decisions and object tracking—required to calibrate the proficiency index. This sprint marked the transition from architectural design to active algorithmic testing, verifying that the input stream provided sufficiently granular data to distinguish between skill levels without introducing latency spikes that would confound the reaction time measurements.

Adaptive Difficulty Algorithm with Theoretical Validation

Sprint 4 was dedicated to the rigorous coding and optimization of the Adaptive Difficulty Engine. The team implemented the Weighted Moving Average (WMA) and Consistency Penalty (CP) calculations using the pre-allocated circular buffer designed in Sprint 1. The critical engineering challenge addressed here was ensuring the mathematical operations executed in constant time O(1), specifically to

counter the battery drain issues associated with deep learning models as noted by IEEE COG (2021). The team wrote the logic to recalculate the Proficiency Index (Phi) at the conclusion of every interaction cycle. Theoretical validation was conducted by simulating erratic user behavior to ensure the "Consistency Penalty" effectively dampened rapid difficulty oscillations, proving the algorithm's stability against noisy data without the need for computationally expensive recurrent neural networks.

Mini-Game Development Batch 2 and Localization with Survey Language Validation

While expanding the content library to generate more diverse data streams, Sprint 5 focused on the storage optimization associated with localization. The team implemented a hash-map based lookup system to ensure that language switching did not incur I/O penalties during runtime. This technically supports the "Green Computing" objective by minimizing storage access operations. Furthermore, the adaptive algorithm underwent parameter tuning based on the initial telemetry from Sprint 3, refining the threshold values in the decision tree to better align with the non-linear learning curves observed in the target data. This tuning process was essential to validate that the rule-based approach could approximate the personalized experience of AI models without the associated computational cost described by Science Direct (2025).

G-Counter CRDT Multiplayer Synchronization with Distributed Systems Validation

The focus of Sprint 6 was the implementation of the G-Counter Conflict-Free Replicated Data Type (CRDT) to solve the problem of network instability described by ISARP (2024). The research team engineered a custom, bare-metal networking stack over UDP. The primary innovation developed during this sprint was the Integer-Only Payload Optimization. Instead of using standard JSON serialization

libraries which produce bloated packets, the team implemented a bit-packed binary protocol. This reduced the synchronization payload to a minimal byte array, directly addressing the high packet loss rates of ad-hoc networks. The sprint culminated in distributed systems validation, where the team simulated network partitions to verify that the mathematical properties of Commutativity, Associativity, and Idempotency successfully guaranteed state convergence upon reconnection

Integration, Refinement, and Post-Implementation Research Documentation

The final sprint focused on the synthesis of the separate modules and the rigorous evaluation of the complete artifact against ISO 25010 standards. The team integrated the adaptive engine and the network manager, resolving thread contention issues to ensure thread-safety between the rendering loop and the background network listener. The final phase involved instrumental profiling to generate the quantitative data for the thesis defense, specifically measuring the battery drain (mAh) and frame rate stability (FPS) to empirically prove the efficiency gains of the O(1) algorithmic approach compared to baseline deep learning models. This phase confirms the system's adherence to the efficiency benchmarks proposed (Paolini et al.)

Integration Pattern Research Artifacts as Prioritized Backlog Items

During Sprint 1 planning the Product Owner translated survey data and interview insights into prioritized backlog items ensuring development aligned with empirical findings. Statistics showing low water scarcity awareness led to specific mini-games while misconceptions about water safety drove the creation of SortWater. Real-world infrastructure constraints identified in interviews inspired realistic household scenarios implemented across Sprints 3 and 5. Additionally a 79% preference for the Filipino language mandated the localization system while climate data informed environmental narratives. Each Sprint Review established a

continuous feedback loop to validate these deliverables. Sprint 3 verified that games like CatchTheRain addressed specific misconceptions and Sprint 5 confirmed linguistic accuracy with experts. Finally Sprint 7 presented a comparative analysis demonstrating that WaterWise successfully bridged the documented knowledge gaps.

See Table 3. in Appendix B for the Table Showing the sprint planning and activities.

Materials

Software Materials

The research utilizes the Godot 4.5 Game Engine as the primary development environment. This platform was selected not for its graphical capabilities, but for its support of GDScript, a pythonic language that allows for rapid prototyping of algorithmic logic while compiling to efficient bytecode for mobile execution. To support the rigorous validation of the algorithms, the team employs Git for version control, allowing for the isolation of specific algorithmic branches, and Visual Studio Code for static code analysis. The choice of these tools aligns with the "Green Computing" methodology by utilizing open-source, lightweight software stacks that do not introduce proprietary bloat to the final artifact.

See Table 4. in Appendix B for the Table showing the softwares selected for the development

See Table 5. in Appendix B for the Table showing the Technology Stack Rationale with Research Support

Hardware Materials

The hardware selection is bifurcated into development and testing environments. Development is conducted on standard workstations to facilitate rapid compilation, utilizing Intel Core i5 or AMD Ryzen 5 processors with 16 GB RAM.

However, the critical research hardware consists of the Target Test Devices: a suite of low-end Android smartphones with specifications as low as 2GB RAM and Quad-core Cortex-A53 processors. These specific devices are required to replicate the thermal throttling and memory pressure conditions that the proposed algorithms are designed to mitigate. Validating the algorithms on high-end hardware would invalidate the research hypothesis regarding efficiency; therefore, the low-spec hardware serves as the primary validation rig for the O(1) complexity claims.

Proposed Algorithm/Approach

This section presents detailed specifications of the two primary algorithmic contributions, the rule-based rolling-window adaptive difficulty algorithm and the G-Counter CRDT multiplayer synchronization mechanism. For each algorithm, we provide mathematical formulations with precise variable definitions, complete pseudocode enabling implementation, flowchart descriptions facilitating conceptual understanding, example walkthroughs demonstrating behavior with sample data, and output parameter specifications.

Rule-Based Rolling-Window Adaptive Difficulty Algorithm

Rolling Window Data Structure. The algorithm utilizes a First-In-First-Out (FIFO) queue data structure of size five, maintaining a rolling window containing the five most recent mini-game performance records while discarding older records that no longer reflect current student capability. This specific window size of five was selected through careful balance of two competing design considerations reflecting fundamental tradeoffs in adaptive system design.

Smaller windows with size less than three enable rapid response to performance changes but create erratic difficulty shifts where students quickly advance and regress repeatedly in oscillating patterns, creating frustration and cognitive instability as students rapidly face changing challenge levels. Larger windows with size greater than seven provide stable performance assessment but

result in sluggish adaptation that frustrates learners whose capabilities have genuinely improved, maintaining overly simplified tasks despite demonstrated mastery.

Contemporary research on dynamic difficulty adjustment demonstrates the effectiveness of rolling window approaches in maintaining optimal engagement, with Lin et al. (2025) finding that rolling window mechanisms with window sizes between 3-7 represent optimal balances between data stability and responsiveness. The size of five represents this empirically-validated balance, providing sufficient data for reliable performance assessment through averaging across multiple games while maintaining responsiveness to genuine capability changes occurring as students learn through gameplay. The rolling window data structure operates through FIFO mechanisms where new performance records are added to the back of the queue and oldest records are automatically removed from the front when the window exceeds size five. Conceptually, the window slides forward with each new game completion, always retaining the five most recent games' data while discarding games from more than five games ago. This design ensures that the assessment basis evolves continuously as students continue gameplay, reflecting current capability rather than being weighted toward historical performance from early in the session when students had less experience. The rolling window approach fundamentally differs from cumulative averaging approaches that weight all historical data equally regardless of recency, recognizing that recent performance provides more valid indicators of current capability than historical performance from the beginning of the session.

Adaptive Difficulty Mechanism and Flow Theory. The adaptive difficulty system is grounded in established psychological principles of optimal engagement and learning. Shohieb et al. (2025) describe how dynamic difficulty adjustment (DDA) automatically adjusts scenarios, parameters, and game behaviors in real time

based on the player's skill level. By comparing a player's current performance to an ideal or reference performance (such as the average first-time performance on a level), the game dynamically modifies difficulty settings. The system maintains optimal balance between challenge and engagement, avoiding boredom when tasks are too easy or frustration when tasks are too difficult.

Circular Buffer Memory Allocation for Rule-Based Rolling Window

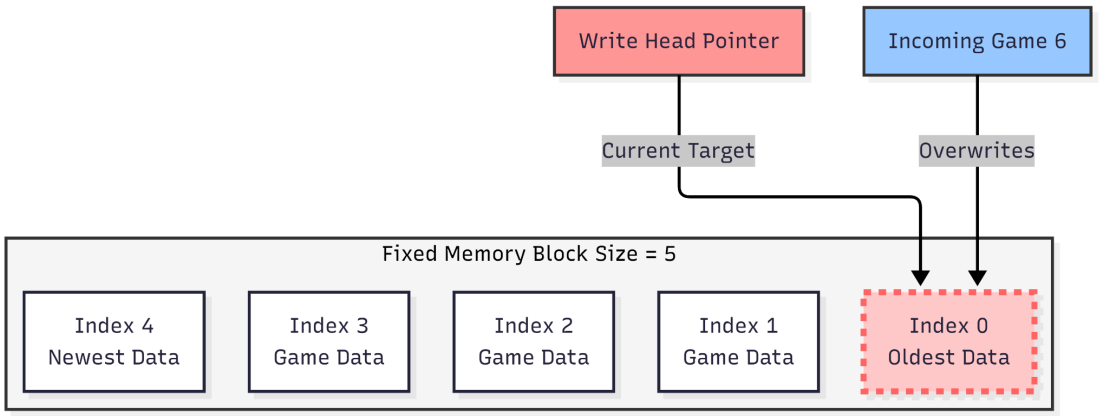


Figure 2. Pseudocode for Rule-Based Rolling-Window Algorithm

The diagram illustrates the memory management strategy employed to achieve (O(1)) Space Complexity for the adaptive difficulty algorithm. Unlike dynamic lists that grow indefinitely with game duration, the system utilizes a fixed-size circular buffer (n=5) pre-allocated in continuous memory. As depicted, the "Write Head Pointer" advances sequentially; when new telemetry (e.g., "Incoming Game 6") arrives, it does not allocate new memory but instead overwrites the oldest data residing at "Index 0." This mechanism guarantees that the algorithm's memory footprint remains constant regardless of session length, effectively preventing heap fragmentation and garbage collection spikes critical optimization factors for preventing thermal throttling on low-end Android devices.

Mathematical Formulation

For each completed mini-game, a composite Raw Game Score is calculated

$$S_i = (w_{acc} \cdot Acc_i) + (w_{speed} \cdot (1 - \frac{RT_i}{T_{max}})) - (w_{err} \cdot E_i)$$

Where

- S_i **= Raw Game Score (0 to 1)**
- w_{acc} **= 0.6 (60% weight for accuracy as primary learning indicator)**
- Acc_i **= 1 for success, 0 for failure**
- w_{speed} **= 0.3 (30% weight for speed indicating fluency development)**
- RT_i **= Reaction time (seconds)**
- T_{max} **= Maximum time allowed (typically 15 seconds)**
- w_{err} **= 0.1 (10% error penalty weight)**
- E_i **= Number of errors committed**

1. **Rolling Average Accuracy**

$$AvgAcc = \frac{\sum_{j=1}^{n} Acc_j}{n} \times 100\%$$

Converted to percentage success rate (0-100%).

2. **Rolling Average Reaction Time**

$$AvgRT = \frac{\sum_{j=1}^{n} RT_j}{n}$$

Measured in seconds for human-interpretable expression.

3. **Rolling Average Reaction Time**

$$WMA = \frac{\sum_{i=1}^{5} w_i \times Acc_i}{\sum_{i=1}^{5} w_i}$$

Where weights $w = [1, 2, 3, 4, 5]$ for 5 most recent games, implementing linear recency bias that prioritizes recent performance.

Example Calculation

Given accuracies [0.60, 0.70, 0.80, 0.90, 0.95]

Weighted Sum = (1×0.60) + (2×0.70) + (3×0.80) + (4×0.90) + (5×0.95) = 12.75

Sum of Weights = 1 + 2 + 3 + 4 + 5 = 15

WMA = 12.75 ÷ 15 = 0.85

This weighting formula prioritizes accuracy (60% weight) as the primary learning indicator, incorporates speed (30% weight) indicating developing fluency, and incorporates error penalties (10%) reflecting that accuracy achieved through many trial-and-error attempts represents lower quality learning than accurate performance with few errors. This approach aligns with contemporary research on game-based assessment. According to Silva et al. (2025), serious games designed for science assessment in primary education demonstrate superior effectiveness when incorporating multiple performance dimensions including accuracy, response time consistency, and error patterns. Their educational design research across 245 primary school children in iterative implementation cycles demonstrated that multidimensional performance assessment captures learning more comprehensively than single-metric evaluation.

1. **Standard Deviation (Consistency Measurement)**

$$\sigma = \sqrt{\frac{\sum_{i=1}^{n}(RT_i - \mu)^2}{n}}$$

n = window size (5)

Where RT_i = individual reaction times, μ = mean

This metric distinguishes between skilled, consistent players and erratic performers who achieve similar averages through luck.

2. **Consistency Penalty**

$$CP = \min\left(\frac{\sigma}{5000}, 0.2\right)$$

Where σ = standard deviation (milliseconds), 5000 = normalization, 0.2 = maximum cap

3. **Proficiency Index (PRIMARY METRIC)**

$$\Phi = WMA - CP$$

Range -0.2 to 1.0

The Proficiency Index represents the player's combined accuracy and consistency performance, serving as the single unified metric for all difficulty decisions.

Rule-Based Decision Tree Logic

Decision Tree Implementation

Three deterministic rules map Φ to difficulty levels

$$\text{Difficulty} = \begin{cases} \text{EASY} & amp; \text{if } \Phi \quad lt; 0.5 \\ \text{MEDIUM} & amp; \text{if } 0.5 \leq \Phi \leq 0.85 \\ \text{HARD} & amp; \text{if } \Phi \quad gt; 0.85 \end{cases}$$

Decision Rules with Research Support

Rule	Condition	Difficulty	Pedagogical Rationale	Actions	Research Support
1	$\Phi < 0.5$	EASY	Struggling/erratic performance	Increase time limits, enable hints, visual guidance	Maintains engagement, prevents frustration (Shohieb et al., 2025)
3	$0.5 \leq \Phi \leq 0.85$	MEDIUM	Optimal Flow State	Standard settings, mild effects	Flow theory foundation (Kas Elias, 2024)
2	$\Phi > 0.85$	HARD	Mastery with consistency	Faster speed, complex tasks, chaos effects	Challenges advanced learners (Silva et al., 2025)

Table 6. Decision Rules with Theoretical and Research Support

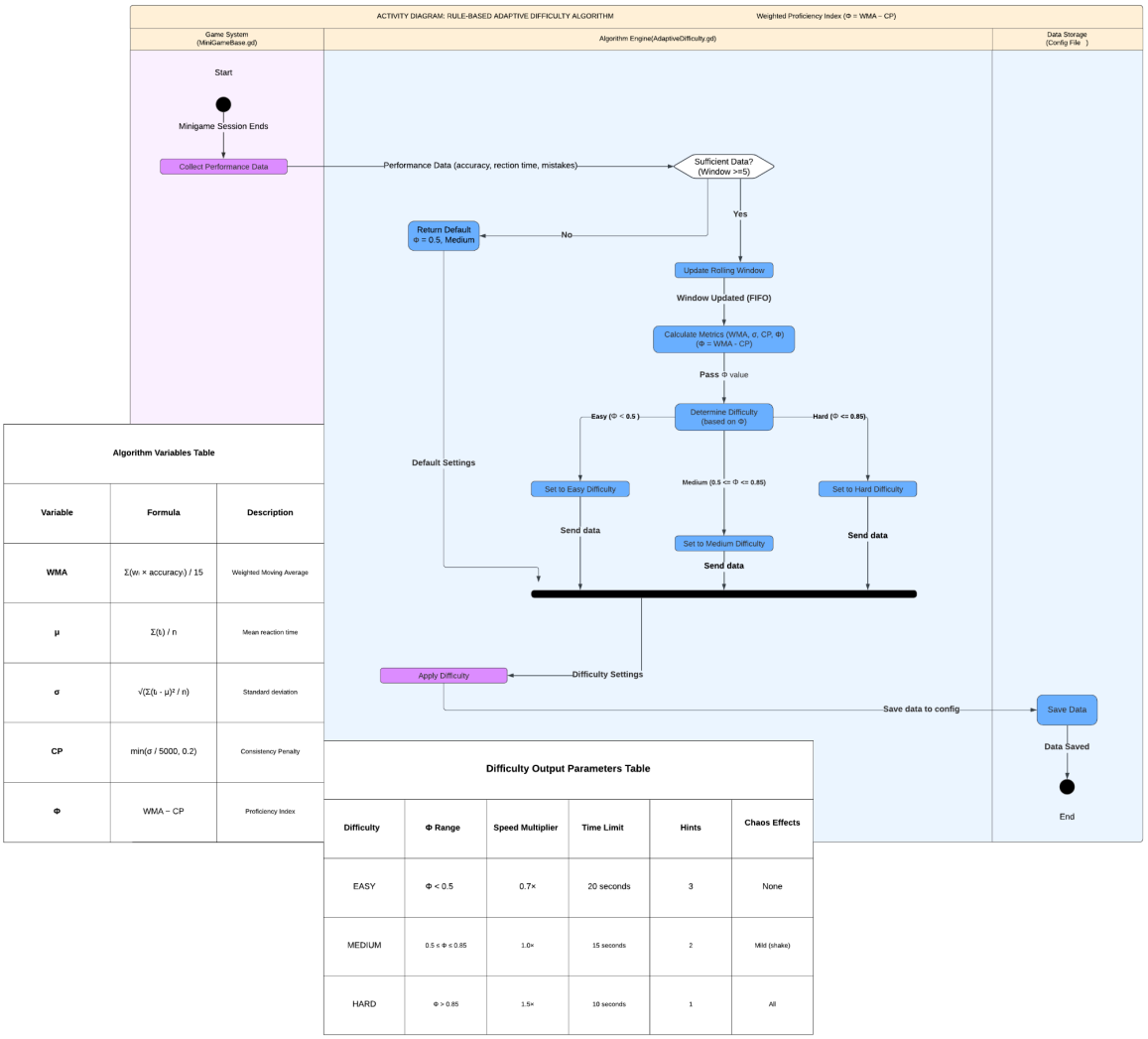
Pseudocode (Rule-Based Rolling-Window Adaptive Difficulty)

```
# Algorithm: Rule-Based Rolling-Window Adaptive Difficulty
# Input: performance_window[5] = [{accuracy, reaction_time, mistakes}]
# Output: DIFFICULTY_SETTINGS {speed_multiplier, time_limit, hints, chaos_effects}

# Step 1: Weighted Moving Average (WMA)
FUNCTION calculate_WMA(performance_window):
    weights = [1, 2, 3, 4, 5]  # Linear recency bias
    weighted_sum = 0
    weight_total = 0

    FOR i FROM 0 TO 4:
        weighted_sum += weights[i] × performance_window[i].accuracy
        weight_total += weights[i]

    WMA = weighted_sum / weight_total
    RETURN WMA

# Step 2: Standard Deviation of Reaction Times
FUNCTION calculate_standard_deviation(performance_window):
    reaction_times = EXTRACT reaction_time FROM performance_window
    mean_time = AVERAGE(reaction_times)
    variance = 0

    FOR each time IN reaction_times:
        variance += (time - mean_time)²

    variance = variance / SIZE(reaction_times)
    std_dev = SQRT(variance)
    RETURN std_dev

# Step 3: Consistency Penalty (CP)
FUNCTION calculate_consistency_penalty(std_dev):
    CP = MIN(std_dev / 5000, 0.2)
    RETURN CP

# Step 4: Proficiency Index (Φ)
FUNCTION calculate_proficiency_index(performance_window):
```

```
    WMA = calculate_WMA(performance_window)
    std_dev = calculate_standard_deviation(performance_window)
    CP = calculate_consistency_penalty(std_dev)

    Φ = WMA - CP
    RETURN Φ

# Step 5: Rule-Based Decision Tree
FUNCTION determine_difficulty(Φ):
    IF Φ &lt; 0.5:
        RETURN DIFFICULTY_SETTINGS{
            speed_multiplier: 0.7,
            time_limit: 20,
            hints: TRUE,
            chaos_effects: NONE,
            visual_guidance: TRUE
        }
    ELSE IF 0.5 ≤ Φ ≤ 0.85:
        RETURN DIFFICULTY_SETTINGS{
            speed_multiplier: 1.0,
            time_limit: 15,
            hints: ON_REQUEST,
            chaos_effects: MILD,
            visual_guidance: TRUE
        }
    ELSE:  # Φ &gt; 0.85
        RETURN DIFFICULTY_SETTINGS{
            speed_multiplier: 1.5,
            time_limit: 10,
            hints: FALSE,
            chaos_effects: STRONG,
            visual_guidance: FALSE
        }

# Main Execution Flow
FUNCTION adaptive_difficulty_main(new_game_performance):
    # Update rolling window (FIFO)
    performance_window.pop_front()  # Remove oldest
    performance_window.push_back(new_game_performance)  # Add newest

    # Calculate proficiency
    Φ = calculate_proficiency_index(performance_window)

    # Determine difficulty settings
    settings = determine_difficulty(Φ)

    # Apply settings to next mini-game
    APPLY(settings)

    RETURN settings
```

Figure 3. Pseudocode for Rule-Based Rolling-Window Algorithm

Activity Diagram of Rule-Based Rolling-Window Algorithm



Algorithm Variables Table

Variable	Formula	Description
WMA	$\Sigma(w_i \times \text{accuracy}_i) / 15$	Weighted Moving Average
μ	$\Sigma(t_i) / n$	Mean reaction time
σ	$\sqrt{(\Sigma(t_i - \mu)^2 / n)}$	Standard deviation
CP	$\min(\sigma / 5000, 0.2)$	Consistency Penalty
Φ	WMA – CP	Proficiency Index

Difficulty Output Parameters Table

Difficulty	Φ Range	Speed Multiplier	Time Limit	Hints	Chaos Effects
EASY	Φ < 0.5	0.7×	20 seconds	3	None
MEDIUM	0.5 ≤ Φ ≤ 0.85	1.0×	15 seconds	2	Mild (shake)
HARD	Φ > 0.85	1.5×	10 seconds	1	All

Figure 4. Adaptive Difficulty Algorithm Diagram

Rule-Based Rolling-Window Adaptive Difficulty Algorithm Simulation

Simulation Scenario - A student plays five consecutive mini-games with the following performance

Game	Accuracy	Reaction Time (ms)	Mistakes
1	0.60	8000	4
2	0.65	7500	3

3	0.75	6500	2
4	0.85	5500	1
5	0.90	5000	0

Table 7. A sample data of the simulation scenario

1. **The rolling window contains all 5 games (window full).**
2. **Calculate WMA**

   Weighted Sum = (1×0.60) + (2×0.65) + (3×0.75) + (4×0.85) + (5×0.90)

   = 0.60 + 1.30 + 2.25 + 3.40 + 4.50 = **12.05**

   Sum of Weights = 1 + 2 + 3 + 4 + 5 = **15**

   WMA = 12.05 / 15 = **0.803**
3. **Calculate Standard Deviation**

   Reaction times [8000, 7500, 6500, 5500, 5000]

   Mean = (8000 + 7500 + 6500 + 5500 + 5000) / 5 = **6500 ms**

   Variance = [(8000-6500)² + (7500-6500)² + (6500-6500)² + (5500-6500)² + (5000-6500)²] / 5

   = [2250000 + 1000000 + 0 + 1000000 + 2250000] / 5 = **1300000**

   σ = √1300000 ≈ **1140 ms**
4. **Calculate CP**

   CP = min(1140 / 5000, 0.2) = min(0.228, 0.2) = 0.2 (capped at maximum)
5. **Calculate Φ**

   Φ = 0.803 - 0.2 = **0.603**
6. **Determine Difficulty**

   Since 0.5 ≤ 0.603 ≤ 0.85, apply **MEDIUM** difficulty for Game 6

Settings = speed_multiplier = 1.0×, time_limit = 15s, hints = on_request, chaos_effects = **mild**

In the simulation the student shows improving accuracy (60% → 90%) and decreasing reaction times (8000ms → 5000ms), indicating skill development. However, the high variability in reaction times (σ = 1140ms) suggests inconsistent performance, possibly due to the learning process. The consistency penalty (CP = 0.2) reduces the proficiency index, appropriately keeping the student in MEDIUM difficulty rather than advancing to HARD. This prevents premature difficulty increases that might frustrate the learner before skills stabilize.

Rule-Based Rolling-Window Adaptive Difficulty Algorithm Output Specification

Output Parameter	Data Type	Range/Values	Description
Φ (Proficiency Index)	Float	[-0.2, 1.0]	Unified performance metric combining accuracy and consistency
WMA (Weighted Moving Average)	Float	[0.0, 1.0]	Recency-weighted average of accuracy scores
CP (Consistency Penalty)	Float	[0.0, 0.2]	Penalty for reaction time variability
Difficulty Level	Enum	{EASY, MEDIUM, HARD}	Categorical difficulty classification
speed_multiplier	Float	{0.7, 1.0, 1.5}	Object movement speed scaling factor
time_limit	Integer	{10, 15, 20}	Seconds allowed for task completion
visual_guidance	Boolean	{TRUE, FALSE}	Whether visual guidance overlays are shown
chaos_effects	Enum	{NONE, MILD, STRONG}	Environmental challenge intensity

Table 8. Algorithm Output Specification for Rule-Based Rolling-Window Adaptive Difficulty Algorithm Specification

G-Counter CRDT Multiplayer Synchronization Algorithm

The G-Counter (Grow-only Counter) algorithm enables conflict-free score synchronization across distributed multiplayer game sessions without requiring

centralized server coordination. This approach addresses a critical challenge in Philippine educational contexts where reliable internet connectivity and server infrastructure may be unavailable or cost-prohibitive.

Conflict-Free Replicated Data Types (CRDTs) represent a class of distributed data structures designed to achieve strong eventual consistency without coordination mechanisms that introduce latency or bottlenecks (Shapiro et al., 2011). CRDTs satisfy two fundamental mathematical properties (1) any replica can be modified independently without coordinating with other replicas, and (2) when any two replicas receive the same set of updates, they deterministically converge to identical states through commutative, associative, and idempotent merge operations. Recent CRDT research (Duncan, 2025; Streaver, 2025) confirms the viability of CRDTs for peer-to-peer multiplayer gaming applications.

The CRDT-based P2P multiplayer game prototype presented by Oak (2024) demonstrated that CRDTs enable real-time collaboration in shared virtual environments without centralized servers, using libp2p for peer-to-peer messaging and Yjs JavaScript library for CRDT implementation. The CRDT Dictionary by Duncan (2025) provides comprehensive analysis of CRDT tradeoffs, noting that G-Counters offer optimal simplicity and performance for monotonically increasing values like game scores.

Mathematical Formulation

The G-Counter maintains a distributed counter array where each player independently increments their own position, with global score computed as the sum across all positions.

Data Structure

Each replica (player device) maintains a counter array

$$\mathrm{g\backslash_counter} = [\mathrm{peer}_1\backslash\mathrm{_score}, \mathrm{peer}_2\backslash\mathrm{_score}]$$

For two-player multiplayer, this becomes

$$\mathrm{g\backslash_counter} = [c_1, c_2]$$

where c1 represents Player 1's score contribution and c2 represents Player 2's score contribution.

Operations

1. **Increment (local operation)**

   Player i increments only their own counter position

$$\mathrm{increment}(i, \mathrm{points}) : c_i \leftarrow c_i + \mathrm{points}$$

This operation is local-only, requiring no network communication. Time complexity o(1)

2. **Query (local operation)**

   The global score is computed as the sum of all counter positions

$$\mathrm{GlobalScore} = \sum_{i=1}^{2} c_i = c_1 + c_2$$

Time complexity $O(n)$ where $n = 2$ players, thus $O(1)$ for fixed player count.

3. **Merge (Synchronization Operation)**

When receiving a remote replica's counter array $\mathrm{g\backslash_counter_{remote}}$, merge by taking element-wise maximum:

$$\mathrm{merge}(\mathrm{g\backslash_counter_{local}}, \mathrm{g\backslash_counter_{remote}}) : c_i \leftarrow \max(c_i^{\mathrm{local}}, c_i^{\mathrm{remote}}) \text{ for } i \in \{1, 2\}$$

Time Complexity $O(n) = O(2) = O(1)$.

The merge operation is

Commutative: $\mathrm{merge}(A, B) = \mathrm{merge}(B, A)$

Associative: $\mathrm{merge}(A, \mathrm{merge}(B, C)) = \mathrm{merge}(\mathrm{merge}(A, B), C)$

Idempotent: $\mathrm{merge}(A, A) = A$

These properties guarantee strong eventual consistency if all replicas receive all updates (regardless of order or timing), they will converge to identical state (Shapiro et al., 2011; Duncan, 2025).

The convergence of the G-Counter CRDT is guaranteed by the mathematical properties of its operations. Given two replicas that have applied a set of increments $\{I_1, I_2 \ldots I_{\square}\}$ in potentially different orders, the system ensures consistency because each increment modifies only a specific counter position 1. The merge process utilizes a maximization operation which is both commutative and associative 2. Consequently, regardless of the sequence in which merges occur, the final state will definitively reflect the maximum increment applied to each position3. Since the system adheres to a grow-only property where increments strictly increase values, this maximum value represents the true count, ensuring that all replicas inevitably converge to an identical state

Vector Merge Logic

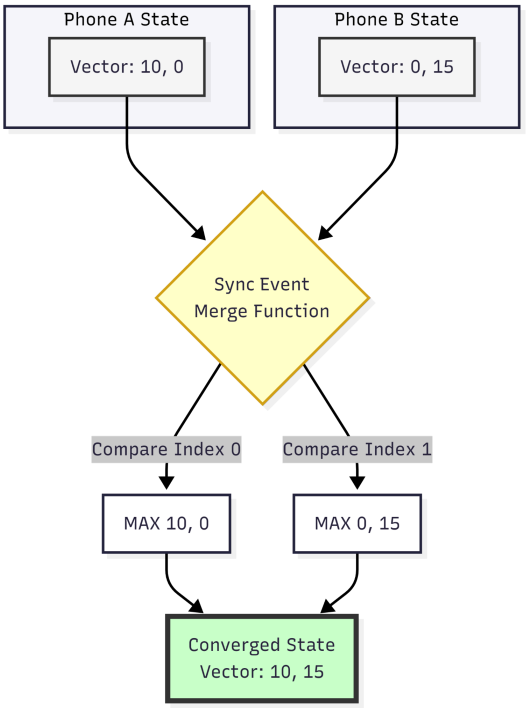


Figure 5. Vector Merge Logic for G-Counter Eventual Consistency

The diagram details the Max-Merge mathematical logic used to resolve state conflicts without a central server, ensuring Strong Eventual Consistency. The diagram presents a partition scenario where "Phone A" records a local score state of [10, 0] while "Phone B" records [0, 15]. Upon a "Sync Event," the G-Counter algorithm executes an element-wise comparison using the MAX() function. By deterministically selecting the highest known value for each index (MAX(10,0) and MAX(0,15)), the system automatically converges both devices to the unified state [10, 15]. This proves that synchronization is commutative and idempotent, guaranteeing that all players effectively reach the same game state regardless of packet arrival order or network latency.

Network Payload Size Comparison

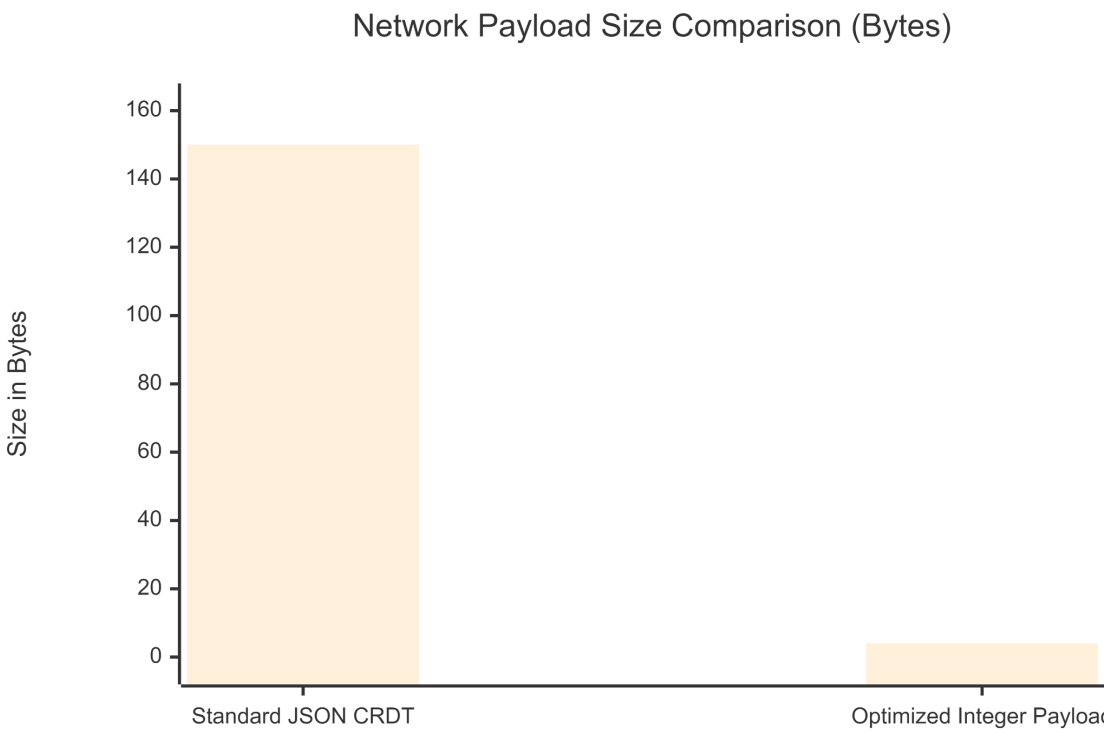


Figure 6. Vector Merge Logic for G-Counter Eventual Consistency

This comparative chart quantifies the bandwidth efficiency gains achieved through the Integer-Only Payload Optimization. The visualization contrasts the

massive overhead of "Standard JSON CRDT" serialization (~150 bytes), which requires verbose string metadata for keys and timestamps, against the proposed "Optimized Integer Payload" (~4 bytes). The substantial differential highlights a 97.3% reduction in data transmission size. This optimization is architecturally significant for the target Mobile Ad-Hoc Network (MANET), as minimizing packet size directly correlates to higher transmission success rates and lower latency jitter in environments characterized by high signal interference and up to 90% packet loss.

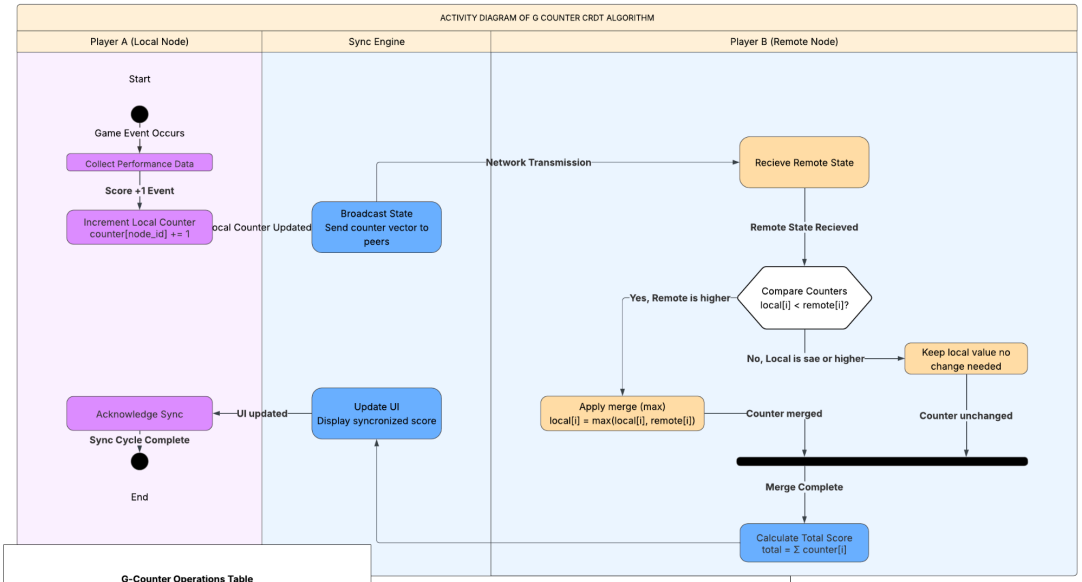
Pseudocode (G-Counter CRDT)

```
# G-Counter CRDT Implementation
# Data Structure: g_counter = [peer_1_score, peer_2_score]

CLASS GCounter:
    ATTRIBUTES:
        peer_id: INTEGER          # This peer's ID (0 or 1)
        g_counter: ARRAY[2]       # Counter array [peer_0, peer_1]

    # Constructor
    FUNCTION __init__(peer_id):
        SELF.peer_id = peer_id
        SELF.g_counter = [0, 0]   # Initialize both counters to 0

    # Increment local counter (when this player scores points)
    FUNCTION increment(points):
        SELF.g_counter[SELF.peer_id] += points
        # Trigger network broadcast of updated counter to peer
        BROADCAST_TO_PEER(SELF.g_counter)

    # Query global score (sum of both players' contributions)
    FUNCTION query_global_score():
        RETURN SELF.g_counter[0] + SELF.g_counter[1]

    # Merge incoming remote counter state
    FUNCTION merge(remote_counter):
        FOR i FROM 0 TO 1:
            # Take element-wise maximum
            SELF.g_counter[i] = MAX(SELF.g_counter[i], remote_counter[i])

        # Update display after merge
        UPDATE_SCORE_DISPLAY()

    # Network receive handler (called when peer counter received)
    FUNCTION on_receive_remote_counter(remote_counter):
        SELF.merge(remote_counter)

# Network Manager Integration
CLASS NetworkManager:
    ATTRIBUTES:
        g_counter: GCounter
        peer_address: STRING
        udp_socket: SOCKET

    # Send local counter to peer
    FUNCTION broadcast_counter():
        message = SERIALIZE(SELF.g_counter.g_counter)
        SEND_UDP(SELF.peer_address, port=7777, message)

    # Receive and process peer counter
    FUNCTION receive_counter():
        message = RECEIVE_UDP(port=7777)
        IF message IS NOT NULL:
            remote_counter = DESERIALIZE(message)
            SELF.g_counter.on_receive_remote_counter(remote_counter)

# Multiplayer Game Flow
FUNCTION multiplayer_game_session():
```

```
# Initialize G-Counter
player_role = DETERMINE_ROLE()  # HOST=0 or CLIENT=1
g_counter = GCounter(peer_id=player_role)

# Game loop
WHILE game_active:
    # Local gameplay
    IF player_action_scored():
        points = CALCULATE_POINTS()
        g_counter.increment(points)

    # Network synchronization (async)
    IF time_since_last_sync &gt; 1000ms:
        BROADCAST_COUNTER()

    # Display synchronized global score
    global_score = g_counter.query_global_score()
    DISPLAY_SCORE(global_score)

# Final synchronization
FINAL_BROADCAST()
WAIT_FOR_FINAL_MERGE()
DISPLAY_FINAL_SCORE()
```

Figure 7. G-Counter Pseudomo Code

Activity Diagram of G-Counter CRDT Algorithm



G-Counter Operations Table

Operation	Formula	Description
Increment	counter[node_id] += 1	Add 1 to local node's counter
Merge	local[i] = max(local[i], remote[i])	Take maximum of each node's counter
Value	Σ counter[i]	Sum all node counters for total

CRDT Properties Table

Property	Definition	Benefit
Commutative	merge(A,B) = merge(B,A)	Order doesn't matter
Associative	merge(merge(A,B),C) = merge(A,merge(B,C))	Grouping doesn't matter
Idempotent	merge(A,A) = A	Duplicate messages safe
Monotonic	Values only increase	No data loss

Figure 8. G-Counter CRDT Diagram

G-Counter CRDT Algorithm Simulation

Simulation - Two players (Host and Client) play cooperatively, each contributing to a shared score.

1. **Initial State**

   Host g_counter = [0, 0]

   Client g_counter = [0, 0]

2. **Event 1 - Host catches rainwater (+5 points)**

   Host executes increment(5)

   Host g_counter = [5, 0]

   Host broadcasts [5, 0] to Client

   Client receives and merges max([0, 0], [5, 0]) = [5, 0]

   Both synchronized [5, 0], GlobalScore = 5

3. **Event 2 - Client fixes leak (+3 points)**

   Client executes increment(3)

   Client g_counter = [5, 3]

   Client broadcasts [5, 3] to Host

   Host receives and merges max([5, 0], [5, 3]) = [5, 3]

   Both synchronized [5, 3], GlobalScore = 8

4. **Event 3 - Concurrent actions (network partition)**

   Host waters plant (+2 points) Host g_counter = [7, 3]

   Client sorts water (+4 points) Client g_counter = [5, 7]

   Divergent state (temporary network delay)

5. **Event 4 Network reconnection and synchronization**

   Host broadcasts [7, 3]

   Client broadcasts [5, 7]

   Both messages eventually received (order doesn't matter due to commutativity)

   Host receives Client's [5,7] merge([7, 3], [5,7]) = [max(7,5), max(3,7)] = [7, 7]

Client receives Host's [7, 3] merge([5, 7], [7, 3]) = [max(5,7), max(7,3)] = [7, 7]

Eventual Consistency Achieved Both [7, 7], GlobalScore = 14

The system demonstrates several robust properties, primarily the fact that no coordination is needed; players can increment values independently without waiting for acknowledgment. This design ensures order independence, guaranteeing that merge results remain identical regardless of the sequence in which messages arrive. Furthermore, the system facilitates automatic reconciliation, allowing divergent states to converge naturally upon synchronization. These features collectively enable strong network partition tolerance, ensuring uninterrupted gameplay even during connectivity issues.

G-Counter CRDT Algorithm Output Specification

Output Parameter	Data Type	Range/Values	Description
g_counter	Array[2]	[0..∞, 0..∞]	Counter array with each player's score contribution
GlobalScore	Integer	[0..∞]	Sum of all counter positions (Σ g_counter[i])
peer_id	Integer	{0, 1}	This replica's player identifier (0=Host, 1=Client)
is_synchronized	Boolean	{TRUE, FALSE}	Whether local and remote counters match (no pending updates)
last_sync_timestamp	Integer	[0..∞]	Milliseconds since last successful merge operation
network_latency	Integer	[0..∞]	Round-trip time for counter synchronization (milliseconds)

Table 9. G-Counter CRDT Algorithm Simulation Specification

System Architecture

The WaterWise system employs a four-module layered architecture implementing separation of concerns, modularity, and clear responsibility boundaries aligned with clean architecture principles. The architecture separates presentation

concerns from business logic, service coordination, and data persistence, facilitating independent development, testing, and modification of each layer.

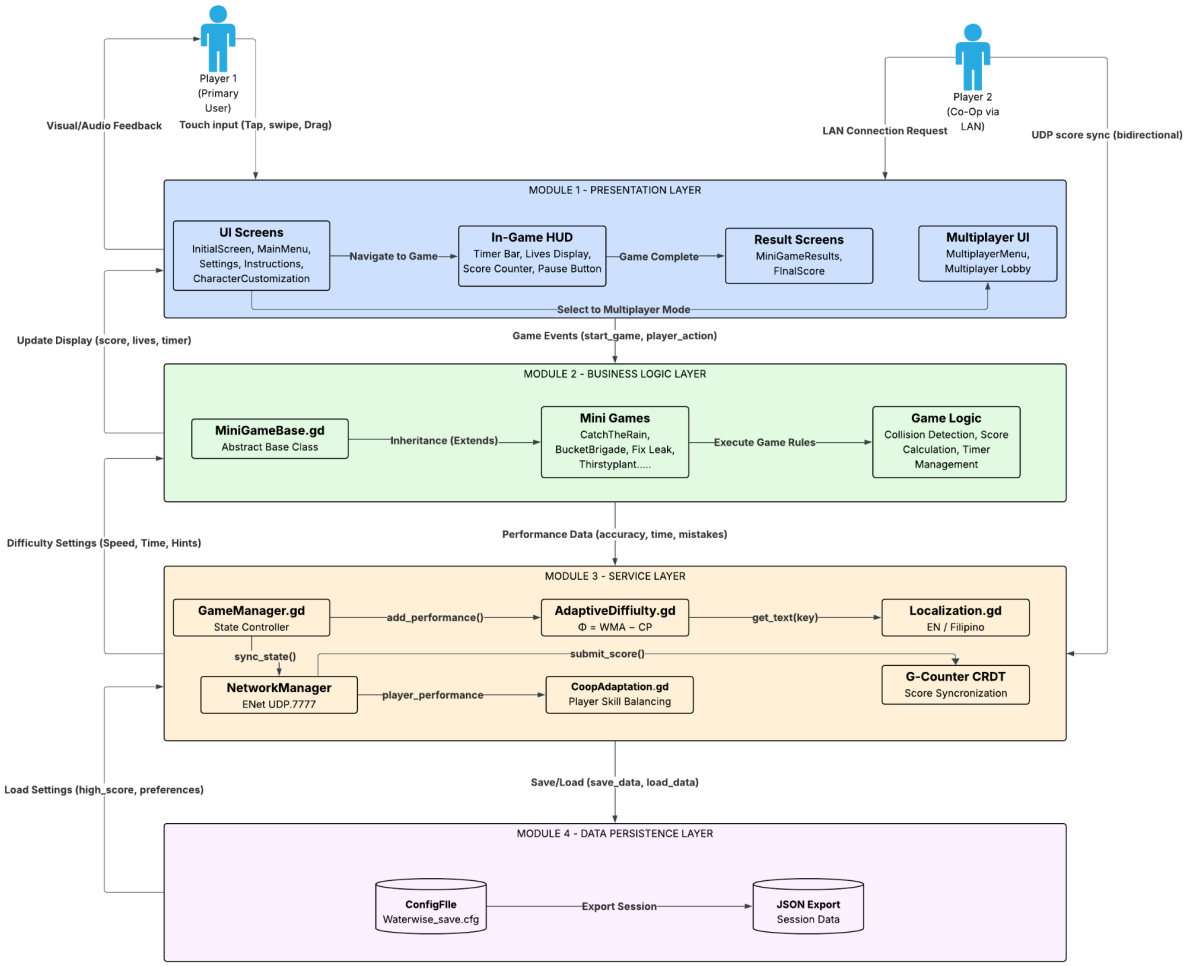


Figure 9. WaterWise Four-Module System Architecture

The system is organized into four distinct modules, each handling specific functional responsibilities to maintain separation of concerns. The Presentation Module governs the visual layer and user interaction, managing interface rendering, input handling, and visual feedback display. Its key components include various UI screens (menus, HUD, results) alongside systems for processing touch or mouse events and managing sprite animations. It triggers game events within the Business Logic Module and relies on it for display updates and feedback, maintaining a strict separation from the lower service layers.

The Business Logic Module encapsulates the core gameplay rules, handling game mechanics implementation, collision detection, score calculation, and mini-game lifecycle management. This module is structured around the MiniGameBase abstract class and includes the 19 specific mini-game implementations and physics systems. It acts as the bridge between the visual layer and the backend services, depending on the Service Module for adaptive difficulty settings and network synchronization data to pass back to the presentation layer.

The Service Module acts as the system's central coordinator, addressing cross-cutting concerns such as state management, adaptive difficulty calculations, networking, and localization. It comprises essential singleton components including the GameManager, AdaptiveDifficulty, NetworkManager, G-Counter, CoopAdaptation, and Localization scripts, and relies on the Data Persistence Module for storage operations and the Business Logic Module for performance data. Finally, the Data Persistence Module manages the permanent storage of configuration files and the export of session data. It utilizes ConfigFile for saving user preferences and generates JSON exports for performance metrics and session logs, operating on data received directly from the Service Module.

Mobile Mesh Topology

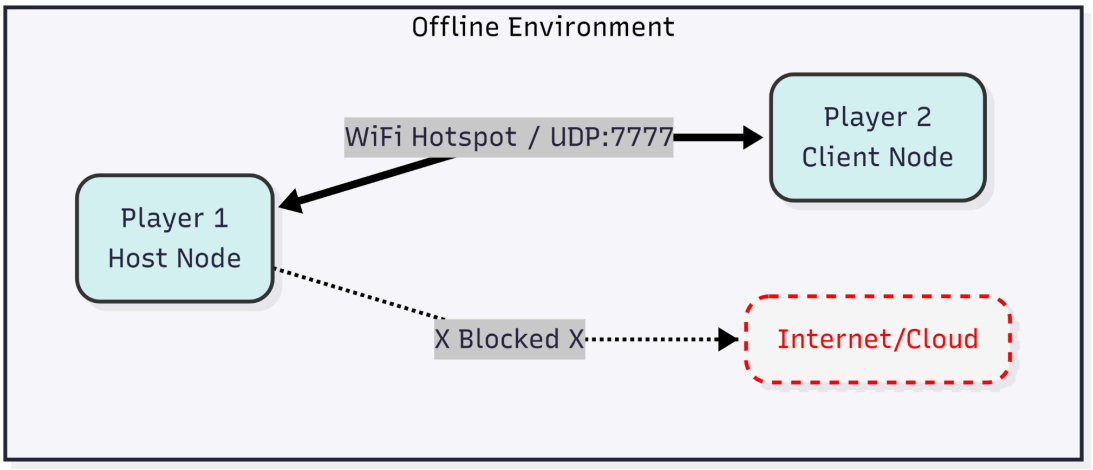


Figure 10. Serverless Mobile Ad-Hoc Network (MANET) Topology

This topology diagram validates the Offline-First architecture of the WaterWise system, specifically designed for disconnected edge environments. The architecture establishes a direct Peer-to-Peer (P2P) link between the "Host Node" (Player 1) and "Client Node" (Player 2) using a WiFi Direct Hotspot channel communicating via UDP Port 7777. Crucially, the diagram explicitly delineates the "Offline Environment" boundary, with a visual disconnect ("X Blocked X") to the Cloud/Internet. This confirms that the system creates a self-contained local mesh capable of strictly lateral synchronization, removing all dependencies on external servers or cellular data, which ensures functionality in remote regions with zero internet availability.

Conceptual Framework

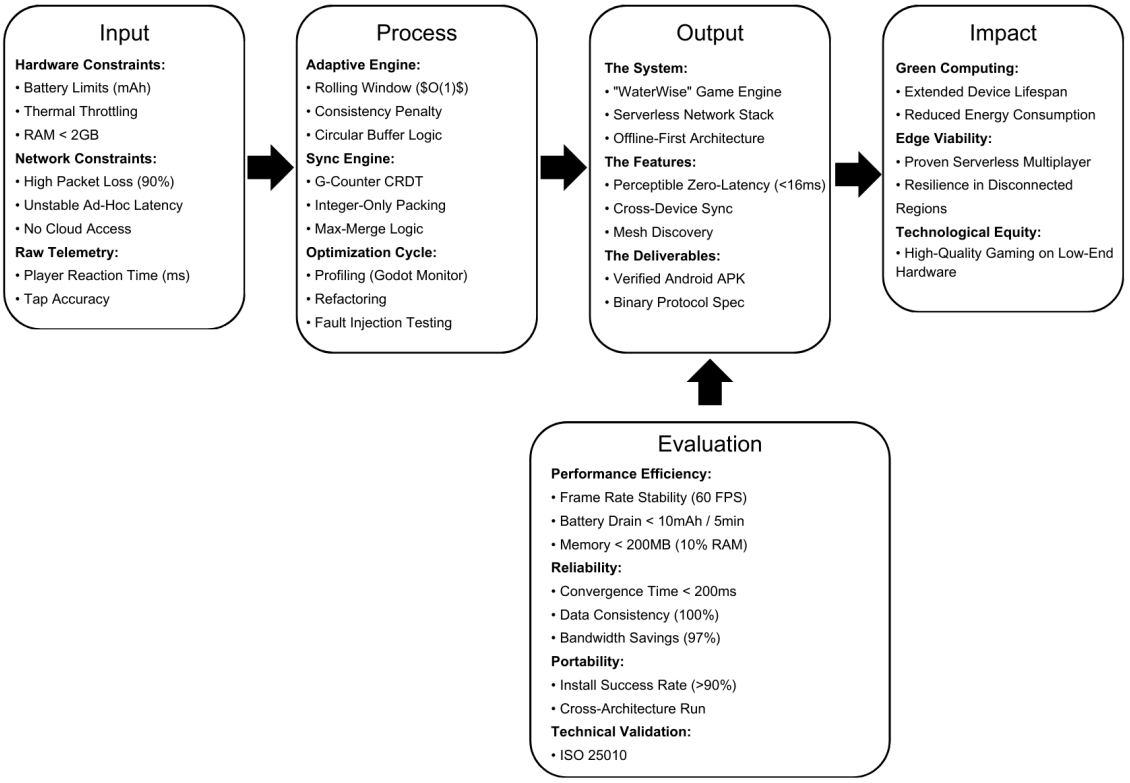


Figure 11. Conceptual framework of Water Wise: A Rule-Based Adaptive Difficulty Game For Water Reuse Education Among Filipino Elementary Students

The study employs an expanded Input-Process-Output-Evaluation-Impact (IPO-EI) model to structure the experimental algorithm engineering of the WaterWise system, establishing a feedback-driven framework for optimizing edge computing applications. The process begins with Input, which rigorously defines the hardware constraints (limited battery, thermal throttling, <2GB RAM) and network instability (90% packet loss, ad-hoc latency) characteristic of the target environment, serving as the foundational variables for architectural design. These constraints feed into the Process stage, where the Adaptive Engine (implementing the O(1) Rule-Based Rolling Window) and the Sync Engine (executing the Integer-Only G-Counter CRDT) are iteratively refined through a cycle of profiling and fault injection testing. This engineering effort yields the Output: a serverless, offline-first game architecture and binary protocol specification capable of perceptible zero-latency (<16ms) adaptation.

The system then undergoes Evaluation, which functions as a formal quality audit against the ISO/IEC 25010 Software Quality Model. Specifically, the system is stress-tested for Performance Efficiency (validating 60 FPS stability and <10mAh battery drain per 5-minute session) and Reliability (confirming convergence times <200ms and data consistency), creating an agile feedback loop that informs further algorithmic tuning. Finally, the framework culminates in Impact, where the validated system contributes to Green Computing through reduced energy consumption and establishes Edge Viability by proving that high-quality, resilient multiplayer gaming is achievable on low-end hardware in disconnected regions.

Use Case Diagram

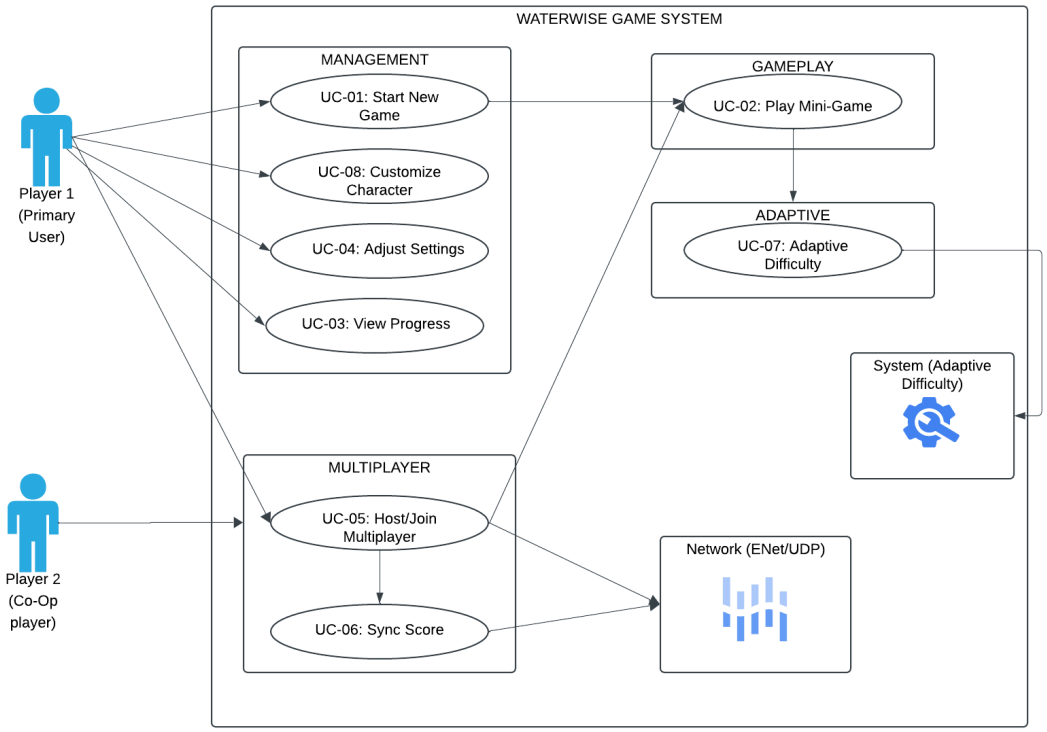


Figure 12. Primary Use Cases Key Use

The system facilitates a comprehensive range of user interactions and background processes, beginning with **UC-01**, where the player initiates a new game session, causing the system to load configurations and present the mini-game selection. To personalize the experience, **UC-08** allows players to customize their

avatar's facial features and accessories, while **UC-04** enables adjustments to global settings such as volume, language, and accessibility. During active gameplay (**UC-02**), the player engages with specific mechanics while the system tracks performance metrics, which are subsequently used in **UC-07** to calculate the Proficiency Index (Φ) and apply the decision tree to adjust parameters for the next game. Players can monitor their long-term development through **UC-03**, which displays cumulative scores, achievements, and difficulty progression. For collaborative scenarios, **UC-05** allows players to host or join LAN sessions via UDP connections, while U**C-06** runs in the background to synchronize scores across devices using G-Counter CRDT merges.

Implementation

Autoload Singleton Registration

Godot's autoload feature registers singleton services globally accessible throughout the application

Singleton Name	Script Path	Purpose
GameManager	autoload/GameManager.gd	State machine coordinating scene transitions, game lifecycle
AdaptiveDifficulty	autoload/AdaptiveDifficulty.gd	Φ calculation, rolling window maintenance, difficulty determination
NetworkManager	autoload/NetworkManager.gd	UDP socket management, peer discovery, packet serialization
Localization	autoload/Localization.gd	Language switching, text translation database
CoopAdaptation	autoload/CoopAdaptation.gd	Skill balancing for multiplayer (load_ratio calculation)

GCounter	autoload/GCounter.gd	CRDT counter array, increment/merge operations
ConfigFile	autoload/ConfigFile.gd	Load/save player preferences to waterwise_save.cfg

Table 10. Autoload Services

These singletons enable any scene to access services via GameManager.transition_to_scene(), AdaptiveDifficulty.calculate_proficiency(), NetworkManager.send_packet(), etc.

Algorithm Integration Sequence

Screen	Description	Key UI Elements
InitialScreen	Entry point showing branding, "Tap to Start"	WaterWise logo, animation (water droplet), touch prompt
MainMenu	Central navigation hub	Single Player button, Multiplayer button, Settings, Instructions, Customize, Progress
SettingsWindow	Configuration panel	Volume sliders (Master, Music, SFX), Language toggle (EN/Filipino), Accessibility options
InstructionsScreen	Gameplay tutorial	Control scheme diagrams, mini-game mechanics explanations, cooperative play strategies

CharacterCustomization	Avatar personalization	Face/body selection, color pickers, accessory options, preview render
MultiplayerLobby	LAN session setup	Host/Join toggle, Player List, Ready-up checkboxes, IP display, Start Game button
InGameHUD	Active gameplay overlay	Timer bar (countdown), Lives icons, Score counter (realtime), Pause button
ResultsScreen	Post-game performance summary	Mini-Game Result (accuracy %, time bonus), Final Score, Difficulty indicator, Next button

Table 11. Prototype Screen Descriptions

The system interface comprises a structured sequence of screens designed to facilitate navigation, configuration, and gameplay. The user experience begins with the InitialScreen, serving as the entry point that displays the WaterWise branding, animated water droplet assets, and a "Tap to Start" prompt. This transitions to the MainMenu, the central navigation hub providing access to Single Player and Multiplayer modes, alongside the Settings, Instructions, Customize, and Progress modules. System configuration is handled via the SettingsWindow, which contains volume sliders for Master, Music, and SFX audio, a toggle for English/Filipino language localization, and accessibility options. To support learner comprehension, the InstructionsScreen functions as a gameplay tutorial, presenting control scheme diagrams, explanations of mini-game mechanics, and strategies for cooperative play.

Personalization and multiplayer interaction are managed through dedicated interfaces. The CharacterCustomization screen enables avatar personalization, allowing users to select facial features, body types, and accessories while viewing a live preview render. For collaborative sessions, the MultiplayerLobby facilitates LAN

session setup, featuring Host/Join toggles, player lists, ready-up checkboxes, IP address displays, and the Start Game button. During active sessions, the InGameHUD provides a real-time overlay displaying the countdown timer bar, remaining lives icons, and the score counter. Finally, the session concludes with the ResultsScreen, which presents a post-game performance summary including mini-game results (accuracy percentage and time bonuses), the final score, the difficulty level indicator, and navigation to the next challenge.

Performance Evaluation

The evaluation methodology focuses on three core dimensions, Algorithmic Correctness (via convergence proofs and complexity analysis), System Responsiveness (via computational efficiency metrics), and Adaptive Behavior (via simulation-based case studies). This approach validates that the technical innovations, specifically the rule-based rolling window and G-Counter CRDT, function as intended under simulated real-world conditions.

Instrumental Profiling and Thermal Stress Testing Protocols

To rigorously quantify the green computing efficiency claims, the study moves beyond gameplay metrics to strictly hardware-level profiling. The evaluation operationalizes thermal throttling not as a subjective feeling of lag, but as a measurable reduction in CPU clock speed triggered by heat accumulation. The device's thermal state is monitored using the Android Profiler and Godot Monitor to log the CPU Temperature (T_{cpu}) in degrees Celsius at 1-second intervals. The primary evaluation metric for throttling is defined as Clock Speed Stability (S_{clk}), where a drop in CPU frequency below 80% of the maximum rated speed constitutes a throttling event.The Rule-Based Algorithm is considered successful if it maintains a CPU temperature below 45 degrees (the standard throttling threshold) over a continuous 30-minute stress test session. Simultaneously, energy consumption metrics are captured by measuring battery drain in milliampere-hours (mAh)

normalized per minute of gameplay. This average discharge rate (ΔE) is compared against a baseline simulation running a standard TensorFlow Lite MobileNet model to represent the Deep Learning alternative. The system must demonstrate a significant reduction in discharge rate to validate the efficiency findings of Jung et al. (2025) regarding heuristic optimizations.

Computational Cost Analysis - Heuristics vs. Deep Learning

This section defines the algorithmic process that ensures frame rate stability by mathematically comparing the instruction count required for decision-making. A standard neural network used for dynamic difficulty adjustment typically requires matrix multiplications for input processing. Even a small network requires approximately O(N^2) floating-point operations (FLOPs) per inference. On low-end ARM Cortex-A53 processors, these continuous FLOPs generate significant heat, leading to the thermal throttling identified by PulseGeek (2025).

In contrast, the proposed Rule-Based Rolling Window algorithm utilizes purely arithmetic operations and conditional logic. The Sliding Window update operates in O(1) time because it involves exactly one array insertion and one deletion, regardless of session length. By replacing quadratic matrix math with constant-time pointer arithmetic, the CPU remains in a low-power state for the majority of the frame duration, waking only to process the lightweight physics engine. This specifically addresses the Frame Rate Stability objective, ensuring the main thread never exceeds the 16.6ms budget required for 60 FPS.

Network Fault Injection Protocol

To evaluate the G-Counter CRDT against the network instability variable, the study employs a fault injection methodology using open-source network conditioner tools. The system is subjected to a controlled "Bad Network" profile mimicking a weak 3G or Hotspot connection by randomly dropping 20% of UDP packets and introducing random latency spikes (jitter) between 100ms and 500ms.The primary

metric for this evaluation is Convergence Time (T_conv), defined as the time elapsed between a client action and the Host state reflecting that action. The G-Counter CRDT is evaluated on its ability to achieve a convergence time of less than 200ms despite the introduced packet loss. This test verifies that the commutative property of the algorithm allows it to bypass the re-transmission overhead that typically slows down standard TCP synchronization in high-loss environments, proving its viability for offline-first multiplayer gaming

Evaluation Metrics

The performance evaluation framework assesses the system across six critical categories using specific metrics, methods, and success criteria. Algorithmic Complexity is evaluated through Big-O analysis of critical operations (time and space complexity), with the specific success criterion that the rolling window update, CRDT merge, and Proficiency Index (**Φ**) calculations all achieve O(1) complexity. Convergence Guarantees focus on the G-Counter's eventual consistency, evaluated through mathematical proofs of CRDT properties, requiring demonstration that commutativity, associativity, and idempotency hold true.In terms of system behavior, Adaptation Accuracy is measured by the correctness of difficulty classifications via simulation with known performance patterns, aiming for >= 90% alignment with expert-validated appropriate difficulty. Responsiveness assesses the latency from performance input to difficulty output using computational timing analysis, requiring less than 100ms adaptation latency to ensure a real-time feel. Network Efficiency examines packet size and synchronization frequency through bandwidth analysis, targeting less than 1 KB per sync packet and a maximum of 1 sync per second. Finally, Frame Rate Stability monitors FPS during adaptive calculations via performance profiling, with the criterion of maintaining >= 60 FPS during **Φ** calculations to ensure smooth gameplay.

See Table 12. in Appendix B for the Table Showing the performance evaluation metrics

Algorithmic Complexity Analysis Adaptive

The computational efficiency of the system is validated through Big-O analysis of its critical operations, demonstrating that both core algorithms operate in constant time. The Adaptive Difficulty Algorithm achieves an overall complexity of O(1), ensuring real-time adaptation without impacting frame rates. Specifically, the rolling window update utilizes a circular buffer implementation to remove the oldest element and add the newest in O(1) time. The mathematical computations for the Weighted Moving Average (WMA) and Standard Deviation involve summing a fixed set of five values (O(5)), which simplifies to O(1) due to the fixed window size . Similarly, the Proficiency Index calculation and Decision Tree evaluation involve simple arithmetic and threshold comparisons, both executing in O(1) time.Parallel efficiency is observed in the G-Counter CRDT multiplayer synchronization algorithm, where all operations maintain O(1) complexity to ensure real-time synchronization. The increment operation consists of a single array index update (O(1)). Both the query operation, which sums two array elements, and the merge operation, which computes the element-wise maximum across two positions, are constant time operations given the fixed two-player limit (O(2) = O(1)). Furthermore, network transmission remains highly efficient, serializing a constant 2-element integer array with a consistent payload size of approximately 8 bytes, ensuring O(1) transmission overhead.

Ethical Considerations

Voluntary Participation and Right to Withdraw

Participation in this research remains entirely voluntary, with parents and guardians retaining complete freedom to decline participation without any negative consequences affecting their child's school grades, access to services, or teacher evaluations. Students who assent to participate retain the right to withdraw at any time during the study without penalty. To ensure true volunteerism, teachers and school administrators facilitate the implementation with explicit guidance to communicate that participation is optional and unrelated to academic performance. These clear communication protocols are designed to prevent the creation of coercive atmospheres where families might feel undue pressure to participate.

Privacy and Data Security

Compliance with the Philippine Data Privacy Act (R.A. 10173) is strictly enforced through principles of transparency, legitimate purpose, and proportionality. Data collection is minimized to essential performance metrics (accuracy, reaction times) required for the educational assessment and adaptive difficulty engine, with no collection of sensitive personal information. Security measures prioritize local-first architecture; data is stored locally on devices using secure encryption, preventing unauthorized third-party access and eliminating data breach risks associated with cloud transmission.

REFERENCES

Algocademy. (2025). When to consider using a circular buffer: A comprehensive guide. https://algocademy.com/blog/when-to-consider-using-a-circular-buffer-a-comprehensive-guide/

Almeida, P. S., Shoker, A., & Baquero, C. (2024). Approaches to conflict-free replicated data types. ACM Computing Surveys, 57(2), 51. https://doi.org/10.1145/3695249

Alphabin. (2025, September 30). Mobile app testing device fragmentation. Alphabin QA Blog. https://www.alphabin.co/blog/mobile-app-testing-device-fragmentation

Asian Development Bank. (2021). Philippines: Water supply and sanitation sector assessment, strategy, and road map. https://www.adb.org/documents/philippines-water-supply-sanitation-sector-assessment-strategy-road-map

Benoit-Cattin, T., Velasco-Montero, D., & Fernández-Berni, J. (2020). Impact of thermal throttling on Long-Term Visual inference in a CPU-Based edge device. Electronics, 9(12), 2106. https://doi.org/10.3390/electronics9122106

Chiotaki, D., Poulopoulos, V., & Karpouzis, K. (2023). Adaptive game-based learning in education: A systematic review. Frontiers in Computer Science, 5, 1062350. https://doi.org/10.3389/fcomp.2023.1062350

Choi, P., Ham, D., Kim, Y., & Kwak, J. (2024). VisionScaling: Dynamic deep learning model and resource scaling in mobile vision applications. IEEE

Internet of Things Journal, 11(8). https://doi.org/10.1109/JIOT.2023.3340277

Da, L., et al. (2024). Extending JSON CRDTs with move operations. Proceedings of the 2024 Conference on Collaborative Document Editing. University of Cambridge. https://www.repository.cam.ac.uk/bitstreams/124d6ba1-0c2b-4d30-9e8c-6541c532ae12/download

Dantas, A., & Baquero, C. (2025). CRDT-based game state synchronization in peer-to-peer VR. arXiv. https://doi.org/10.48550/arXiv.2503.17826

Datar, M., Gionis, A., Indyk, P., & Motwani, R. (2002). Maintaining stream statistics over sliding windows. SIAM Journal on Computing, 31(6), 1794-1813. https://doi.org/10.1137/S0097539701398363

Dubey, A., et al. (2024). On-device language models: A comprehensive review. arXiv. https://doi.org/10.48550/arXiv.2409.00088

Escultor, M. (2022). Ugaling Pinoy: A mobile learning game application to enhance learning in Values subject of elementary pupils. International Journal of Educational Research & Social Sciences, 3(3). https://doi.org/10.51601/IJERSC.V3I3.401

GIANTY. (2025). What game porting is and how to successfully port your game. https://www.gianty.com/game-porting-and-how-to-port-your-game/

Godot Engine Forum. (2025). Now I'm worried: Performance on Android [Online forum post]. https://forum.godotengine.org/t/now-im-worried-performance-on-android/124716

Gómez Niño, A., et al. (2024). Gamifying learning with AI: A pathway to 21st-century skills. Journal of Education. https://www.tandfonline.com/doi/full/10.1080/02568543.2024.2421974

Guidec, F., Mahéo, Y., & Noûs, C. (2022). Supporting conflict-free replicated data types in opportunistic networks. Peer-to-Peer Networking and Applications, 15, 1234–1250. https://link.springer.com/article/10.1007/s12083-022-01404-6

Intelivita. (2025). What is Android OS? Version history, features, architecture, and comparisons. https://www.intelivita.com/blog/android-os/

Jiang, X. (2012). A feasibility study of using peer-to-peer architecture to support massively multiplayer online role playing games. https://hdl.handle.net/10779/uow.27664293

Jung, D., Lee, J., Jeong, H., Cha, D., & Kim, H. (2025). Split computing for mobile devices: Energy and latency perspective. IEEE Transactions on Services Computing. https://www.computer.org/csdl/journal/sc/2025/03/10978101/26htiqpLM9W

Kang, S., Choi, H., Park, S., Park, C., Lee, J., Lee, U., & Lee, S. J. (2019, August). Fire in your hands: Understanding thermal behavior of smartphones. In The 25th Annual International Conference on Mobile Computing and Networking (pp. 1-16).

Kim, Y., Lee, J., Jeong, H., Cha, D., & Kim, H. (2021). zTT: Learning-based DVFS with zero thermal throttling for mobile devices. Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services, 1–13. https://doi.org/10.1145/3458864.3468161

Kroupp, G. (2024, July 22). Mastering multiplayer game architecture: Choosing the right approach. Getgud.io. https://www.getgud.io/blog/mastering-multiplayer-game-architecture-choosing-the-right-approach/

Li, Z., et al. (2024). A benchmark for ML inference latency on mobile devices. Proceedings of the 2024 Workshop on Edge Systems, Analytics and Networking (EdgeSys). https://qed.usc.edu/paolieri/papers/2024_edgesys_mobile_inference_benchmark.pdf

Liu, L., et al. (2025). Optimizing lightweight neural networks for efficient mobile edge computing. Scientific Reports, 12(4), e4652. https://doi.org/10.1038/s41598-025-04652-7

Lopes, P., Fachada, N., & Fonseca, M. (2025). Closing the loop: A systematic review of experience-driven game adaptation. arXiv. https://doi.org/10.48550/arXiv.2505.01351

Maghsoudnia, A., et al. (2024). Ultra-reliable low-latency in 5G: A close reality or a vision? Proceedings of the ACM SIGCOMM 2024 Conference on Hot Topics in Networks (HotNets). https://conferences.sigcomm.org/hotnets/2024/papers/hotnets24-56.pdf

Marmat, M., & Thankachan, B. (2025). ML-driven latency optimization for mobile edge computing in fiber–wireless access networks. MethodsX, 15, 103594. https://doi.org/10.1016/j.mex.2025.103594

Mees, M., Jay, T., & Habgood, J. (2018). Designing an adaptive learner model for a mathematics game. Proceedings of the 12th European Conference on Games Based Learning, 800–807. https://shura.shu.ac.uk/22449/

National Economic and Development Authority. (2023). Philippine development plan 2023-2028: Accelerating infrastructure development (Water Resources). Republic of the Philippines. https://pdp.neda.gov.ph/

Or, D. B., et al. (2021). DL-DDA: Deep learning based dynamic difficulty adjustment. Proceedings of the 2021 IEEE Conference on Games (CoG). https://ieee-cog.org/2021/assets/papers/paper_142.pdf

Paolini, E., et al. (2024). Efficient distributed learning over lossy wireless networks. 2024 20th International Conference on Network and Service Management (CNSM). https://dl.ifip.org/db/conf/cnsm/cnsm2024/1571047763.pdf

Philippine Water Works Association. (2025, October 28). PWWA warns of water crisis by 2030. SunStar Davao. https://www.sunstar.com.ph/davao/pwwa-warns-of-water-crisis-by-2030

Plass, J. L., Homer, B. D., & Kinzer, C. K. (2020). Foundations of game-based learning. Educational Psychologist, 50(4), 258-283.

Preguiça, N., Baquero, C., & Shapiro, M. (2018). Conflict-free replicated data types (CRDTs). In Encyclopedia of Big Data Technologies (pp. 1–10). Springer. https://doi.org/10.1007/978-3-319-63962-8_185-1

Promwad. (2025). The trend of affordable AI models and on-device inference. https://promwad.com/news/energy-efficiency-affordable-ai-on-device-inference/

Qubit Labs. (2024). What is green computing? Examples of green computing. https://qubit-labs.com/examples-of-green-computing/

Quinnox. (2025). What is Android fragmentation and how to deal with it. https://www.quinnox.com/blogs/what-is-android-fragmentation/

Rinberg, R., Keidar, I., & Basin, D. (2022). Fast and consistent conflict-free replicated data types. IEEE Transactions on Parallel and Distributed Systems, 33(11), 2635-2647.

Rua, R., & Saraiva, J. (2023). A large-scale empirical study on mobile performance: Energy, run-time and memory. Empirical Software Engineering, 28(2), Article 35. https://doi.org/10.1007/s10664-022-10275-5

Sahin, C., et al. (2024). Do popular apps have issues regarding energy efficiency? JMIR mHealth and uHealth. https://pmc.ncbi.nlm.nih.gov/articles/PMC10909214/

Sahin, O., & Coskun, A. K. (2015). On the impacts of greedy thermal management in mobile devices. IEEE Embedded Systems Letters, 7(2), 55-58. https://www.bu.edu/peaclab/files/2015/05/sahin_ESL15.pdf

Samsung. (2024). Use game optimization to extend battery life during mobile gaming. Samsung Support. https://www.samsung.com/us/support/galaxy-battery/optimization/

Saquib, N., Krintz, C., & Wolski, R. (2022). Log-based CRDT for edge applications. 2022 IEEE International Conference on Cloud Engineering (IC2E), 126–137. https://doi.org/10.1109/IC2E55432.2022.00021

Shapiro, M., et al. (2025). About CRDTs. CRDT.tech. https://crdt.tech

Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-free replicated data types. Proceedings of the 15th International Symposium on Stabilization, Safety, and Security of Distributed Systems, 386-400.

Shohieb, S. M., Bakhit, S. H., Mohammed, E., & Elhady, A. M. (2025). Empowering children with Down syndrome by enhancing emergency preparedness through serious games: Quasi-experimental study with a between-group design. JMIR Serious Games, 13, e73690. https://doi.org/10.2196/56382

Sypytkowski, B. (2021). Optimizing state-based CRDTs (Part 1). Bartosz Sypytkowski. https://www.bartoszsypytkowski.com/optimizing-state-based-crdts-1/

Team, T. C. (2025, November 11). The risks of Android OS fragmentation in enterprises. Trio Mobile Device Management. https://www.trio.so/blog/android-os-fragmentation

Tsai, S. K., Chuang, T. Y., & Lu, Y. H. (2025). Technology-enhanced digital game-based learning for environmental literacy: Catalyzing attitude change in learners. Frontiers in Education, 10, 1629670. https://doi.org/10.3389/feduc.2025.1629670

Tschudin, C. (2022). A connectionless grow-only set CRDT. Proceedings of the 23rd International Middleware Conference (DICG '22). https://dl.acm.org/doi/10.1145/3565383.3566110

Upadhya, K. J., Lobo, R., Chhabra, M. S., Paleja, A., Rao, B. D., M, G., Sisodia, P., & Reddy, B. A. (2025). Sliding window based rare partial periodic pattern mining algorithms over temporal data streams. Frontiers in Big Data, 8, 1600267. https://doi.org/10.3389/fdata.2025.1600267

Velásquez, E. A. P., Paragarino, V. R., & Navarro, D. S. (2024). Developing learning skills through game-based learning in complex scenarios: A case in undergraduate logistics education. Journal of Technology and Science Education, 14(1), 169-183.

Viitanen, H. (2025). Deterministic and synchronous computation between client and server in mobile games (Unpublished master's thesis). Metropolia University of Applied Sciences.

Yan, L., et al. (2024). Mob-FGSR: Frame generation and super resolution for mobile real-time rendering. Proceedings of ACM SIGGRAPH Conference Papers '24. https://sites.cs.ucsb.edu/~lingqi/publications/paper_sig24mobfgsr.pdf

Yahyavi, A., & Kemme, B. (2013). Peer-to-peer architectures for massively multiplayer online games: A survey. ACM Computing Surveys, 46(1), 1–51. https://doi.org/10.1145/2522968.2522977

Ye, H., Zhang, H., Chen, Q., Sun, B., & Wen, B. (2025). MMORPG-based collaborative design and construction simulation in prefabricated buildings. Journal of Asian Architecture and Building Engineering, 24(3), 1518-1534.

Zarghani, A., & Abdedi, S. (2025). Designing adaptive algorithms based on reinforcement learning for dynamic optimization of sliding window size in multi-dimensional data streams. arXiv. https://doi.org/10.48550/arXiv.2507.06901

Zhang, X., Zhang, X., & Yang, W. (2022). Joint offloading and resource allocation using deep reinforcement learning in mobile edge computing. IEEE Transactions on Network Science and Engineering. https://ieeexplore.ieee.org/document/9802709

APPENDICES

Appendix 1

Figures

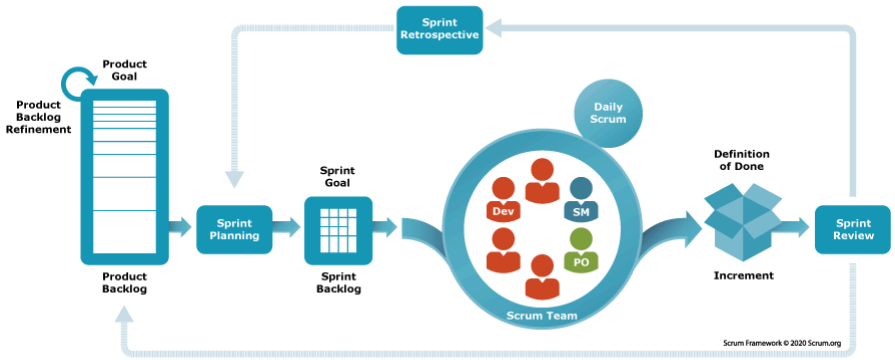
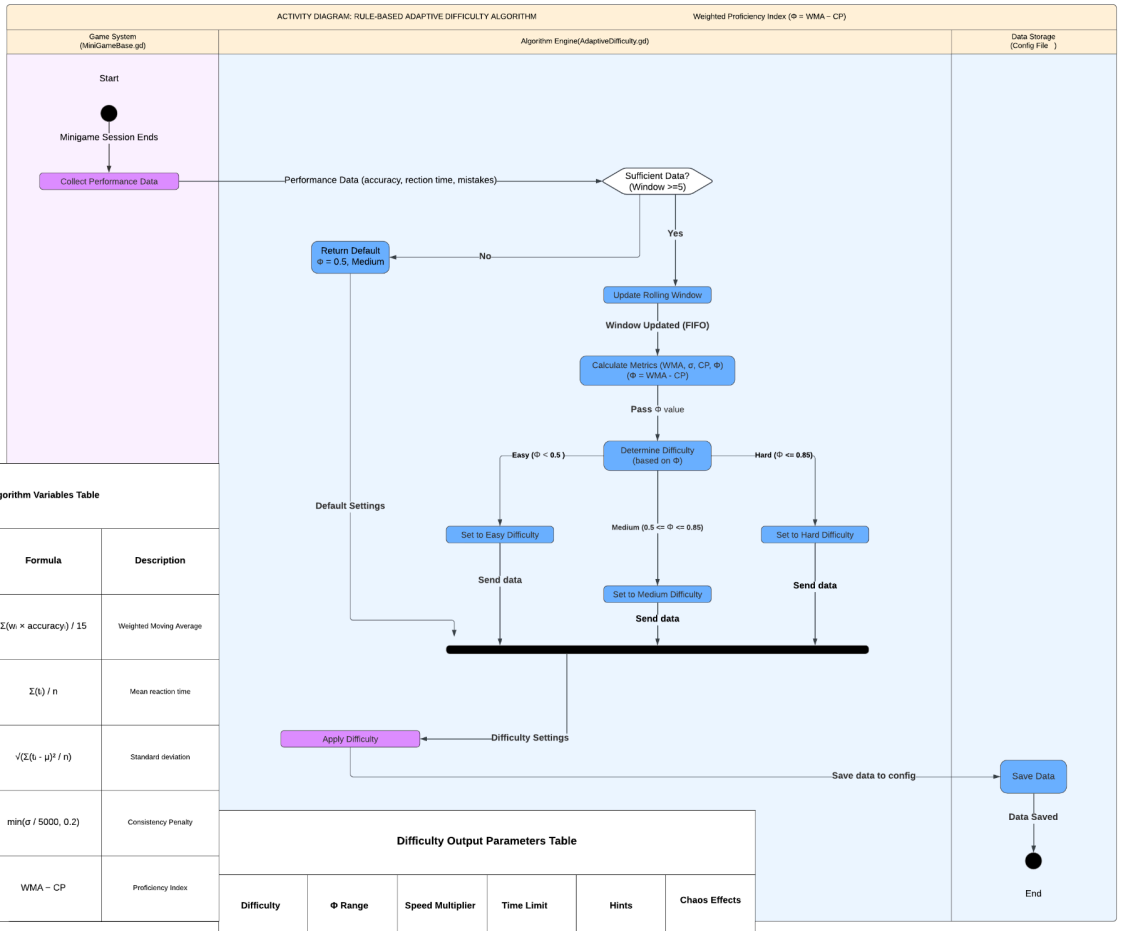


Figure 1. The Scrum framework (Scrum.org, n.d.)

```
# Algorithm: Rule-Based Rolling-Window Adaptive Difficulty
# Input: performance_window[5] = [{accuracy, reaction_time, mistakes}]
# Output: DIFFICULTY_SETTINGS {speed_multiplier, time_limit, hints, chaos_effects}

# Step 1: Weighted Moving Average (WMA)
FUNCTION calculate_WMA(performance_window):
    weights = [1, 2, 3, 4, 5]  # Linear recency bias
    weighted_sum = 0
    weight_total = 0

    FOR i FROM 0 TO 4:
        weighted_sum += weights[i] × performance_window[i].accuracy
        weight_total += weights[i]

    WMA = weighted_sum / weight_total
    RETURN WMA

# Step 2: Standard Deviation of Reaction Times
FUNCTION calculate_standard_deviation(performance_window):
    reaction_times = EXTRACT reaction_time FROM performance_window
    mean_time = AVERAGE(reaction_times)
    variance = 0

    FOR each time IN reaction_times:
        variance += (time - mean_time)²

    variance = variance / SIZE(reaction_times)
    std_dev = SQRT(variance)
    RETURN std_dev

# Step 3: Consistency Penalty (CP)
FUNCTION calculate_consistency_penalty(std_dev):
    CP = MIN(std_dev / 5000, 0.2)
    RETURN CP

# Step 4: Proficiency Index (Φ)
FUNCTION calculate_proficiency_index(performance_window):
```

```
    WMA = calculate_WMA(performance_window)
    std_dev = calculate_standard_deviation(performance_window)
    CP = calculate_consistency_penalty(std_dev)

    Φ = WMA - CP
    RETURN Φ

# Step 5: Rule-Based Decision Tree
FUNCTION determine_difficulty(Φ):
    IF Φ &lt; 0.5:
        RETURN DIFFICULTY_SETTINGS{
            speed_multiplier: 0.7,
            time_limit: 20,
            hints: TRUE,
            chaos_effects: NONE,
            visual_guidance: TRUE
        }
    ELSE IF 0.5 ≤ Φ ≤ 0.85:
        RETURN DIFFICULTY_SETTINGS{
            speed_multiplier: 1.0,
            time_limit: 15,
            hints: ON_REQUEST,
            chaos_effects: MILD,
            visual_guidance: TRUE
        }
    ELSE:  # Φ &gt; 0.85
        RETURN DIFFICULTY_SETTINGS{
            speed_multiplier: 1.5,
            time_limit: 10,
            hints: FALSE,
            chaos_effects: STRONG,
            visual_guidance: FALSE
        }

# Main Execution Flow
FUNCTION adaptive_difficulty_main(new_game_performance):
    # Update rolling window (FIFO)
    performance_window.pop_front()  # Remove oldest
    performance_window.push_back(new_game_performance)  # Add newest

    # Calculate proficiency
    Φ = calculate_proficiency_index(performance_window)

    # Determine difficulty settings
    settings = determine_difficulty(Φ)

    # Apply settings to next mini-game
    APPLY(settings)

    RETURN settings
```

Figure 2. Pseudocode for Rule-Based Rolling-Window Algorithm



Algorithm Variables Table

Variable	Formula	Description
WMA	$\Sigma(w_i \times \text{accuracy}_i) / 15$	Weighted Moving Average
μ	$\Sigma(t_i) / n$	Mean reaction time
σ	$\sqrt{(\Sigma(t_i - \mu)^2 / n)}$	Standard deviation
CP	$\min(\sigma / 5000, 0.2)$	Consistency Penalty
Φ	WMA − CP	Proficiency Index

Difficulty Output Parameters Table

Difficulty	Φ Range	Speed Multiplier	Time Limit	Hints	Chaos Effects
EASY	$\Phi < 0.5$	0.7×	20 seconds	3	None
MEDIUM	$0.5 \le \Phi \le 0.85$	1.0×	15 seconds	2	Mild (shake)
HARD	$\Phi > 0.85$	1.5×	10 seconds	1	All

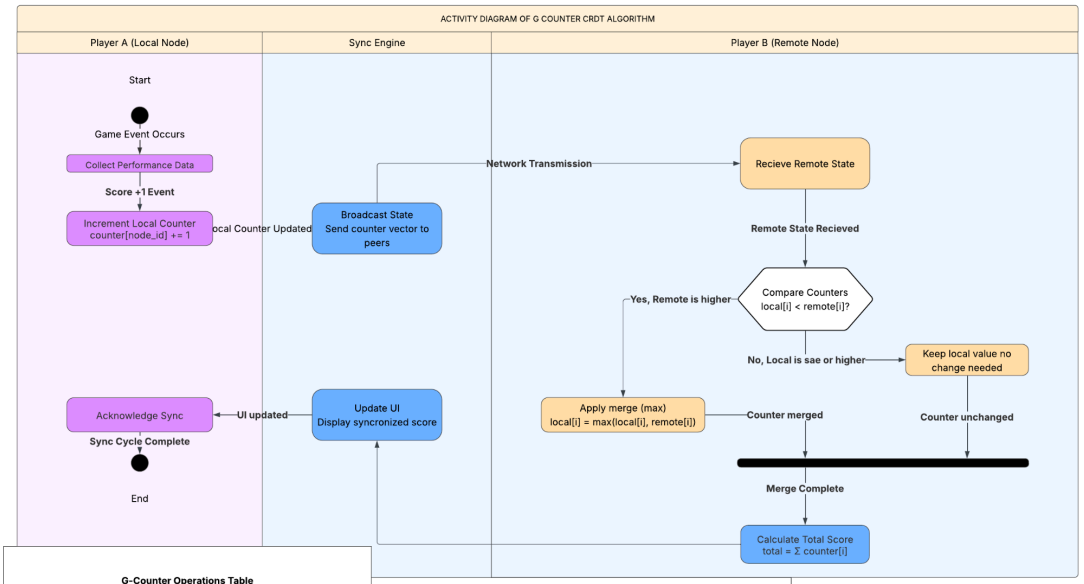
Figure 3. Adaptive Difficulty Algorithm Diagram

```
# G-Counter CRDT Implementation
# Data Structure: g_counter = [peer_1_score, peer_2_score]

CLASS GCounter:
    ATTRIBUTES:
        peer_id: INTEGER          # This peer's ID (0 or 1)
        g_counter: ARRAY[2]       # Counter array [peer_0, peer_1]

    # Constructor
    FUNCTION __init__(peer_id):
        SELF.peer_id = peer_id
        SELF.g_counter = [0, 0]   # Initialize both counters to 0

    # Increment local counter (when this player scores points)
    FUNCTION increment(points):
        SELF.g_counter[SELF.peer_id] += points
        # Trigger network broadcast of updated counter to peer
        BROADCAST_TO_PEER(SELF.g_counter)

    # Query global score (sum of both players' contributions)
    FUNCTION query_global_score():
        RETURN SELF.g_counter[0] + SELF.g_counter[1]

    # Merge incoming remote counter state
    FUNCTION merge(remote_counter):
        FOR i FROM 0 TO 1:
            # Take element-wise maximum
            SELF.g_counter[i] = MAX(SELF.g_counter[i], remote_counter[i])

        # Update display after merge
        UPDATE_SCORE_DISPLAY()

    # Network receive handler (called when peer counter received)
    FUNCTION on_receive_remote_counter(remote_counter):
        SELF.merge(remote_counter)

# Network Manager Integration
CLASS NetworkManager:
    ATTRIBUTES:
        g_counter: GCounter
        peer_address: STRING
        udp_socket: SOCKET

    # Send local counter to peer
    FUNCTION broadcast_counter():
        message = SERIALIZE(SELF.g_counter.g_counter)
        SEND_UDP(SELF.peer_address, port=7777, message)

    # Receive and process peer counter
    FUNCTION receive_counter():
        message = RECEIVE_UDP(port=7777)
        IF message IS NOT NULL:
            remote_counter = DESERIALIZE(message)
            SELF.g_counter.on_receive_remote_counter(remote_counter)

# Multiplayer Game Flow
FUNCTION multiplayer_game_session():
```

57

```
    # Initialize G-Counter
    player_role = DETERMINE_ROLE()  # HOST=0 or CLIENT=1
    g_counter = GCounter(peer_id=player_role)

    # Game loop
    WHILE game_active:
        # Local gameplay
        IF player_action_scored():
            points = CALCULATE_POINTS()
            g_counter.increment(points)

        # Network synchronization (async)
        IF time_since_last_sync &gt; 1000ms:
            BROADCAST_COUNTER()

        # Display synchronized global score
        global_score = g_counter.query_global_score()
        DISPLAY_SCORE(global_score)

    # Final synchronization
    FINAL_BROADCAST()
    WAIT_FOR_FINAL_MERGE()
    DISPLAY_FINAL_SCORE()
```

Figure 4. G-Counter Pseudomo Code



G-Counter Operations Table

Operation	Formula	Description
Increment	counter[node_id] += 1	Add 1 to local node's counter
Merge	local[i] = max(local[i], remote[i])	Take maximum of each node's counter
Value	Σ counter[i]	Sum all node counters for total

CRDT Properties Table

Property	Definition	Benefit
Commutative	merge(A,B) = merge(B,A)	Order doesn't matter
Associative	merge(merge(A,B),C) = merge(A,merge(B,C))	Grouping doesn't matter
Idempotent	merge(A,A) = A	Duplicate messages safe
Monotonic	Values only increase	No data loss

Figure 5. G-Counter CRDT Diagram

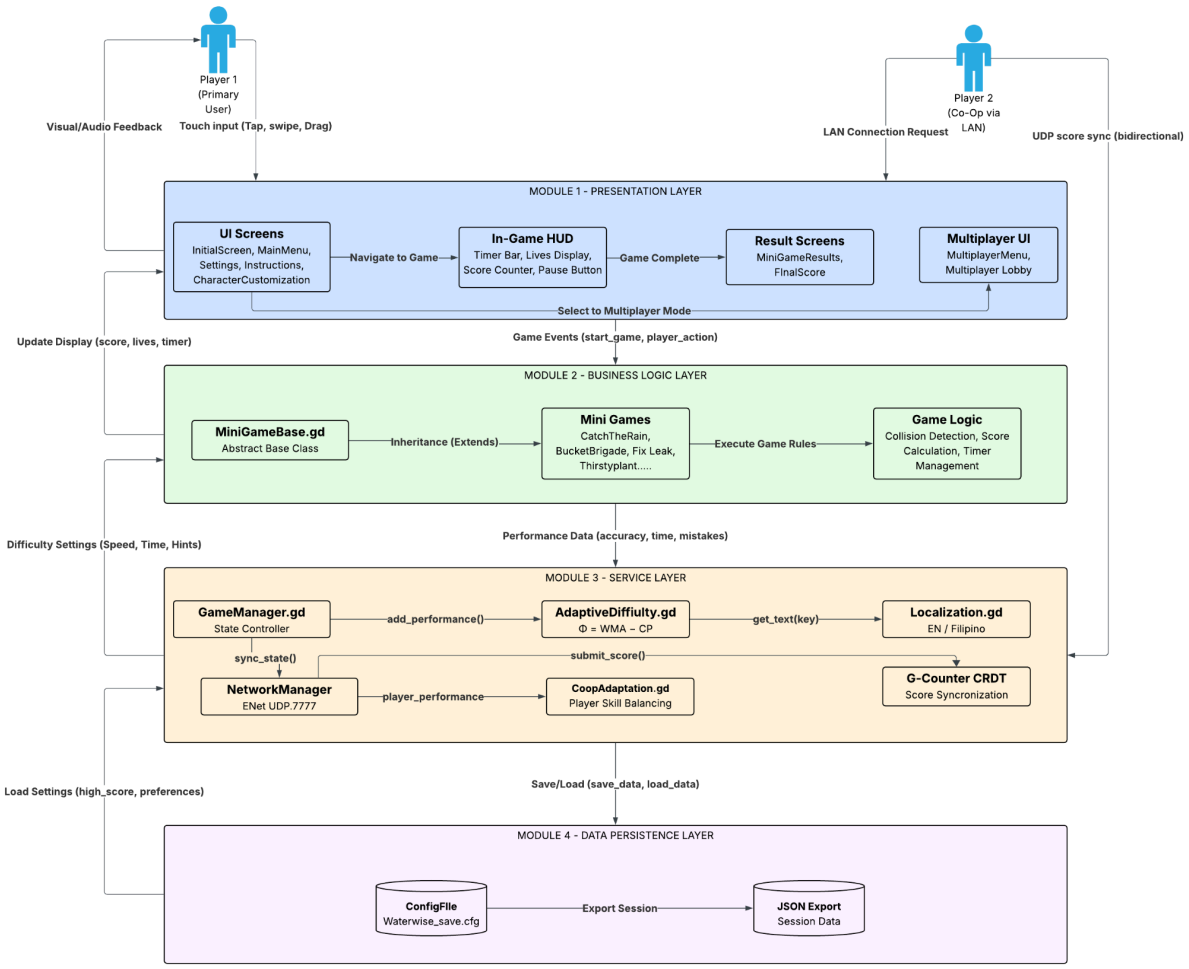


Figure 6. WaterWise Four-Module System Architecture

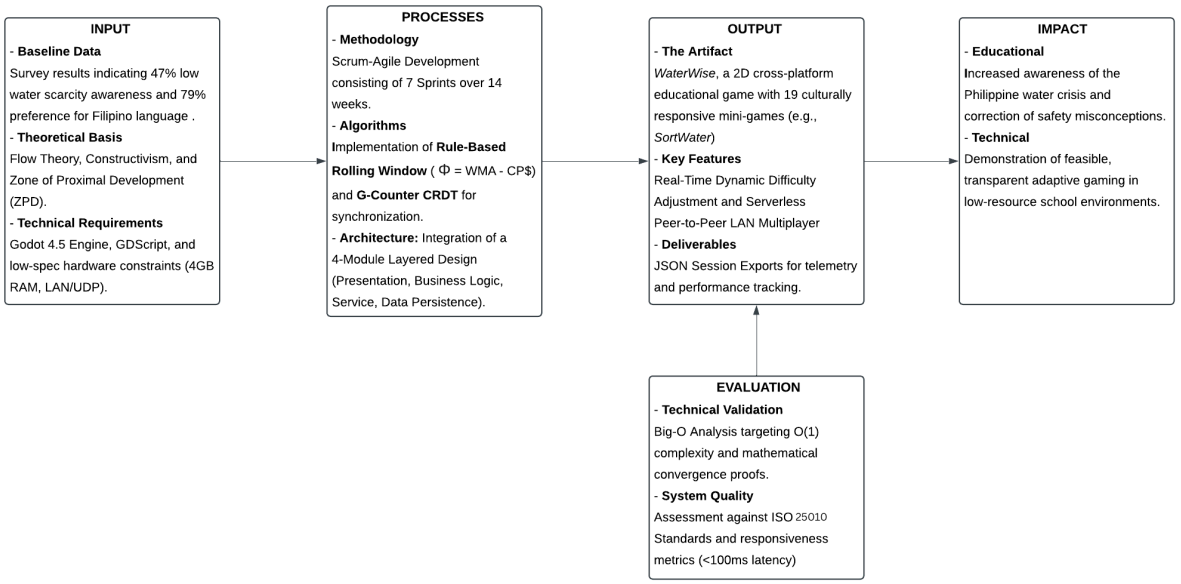


Figure 7. System-Level IPO Model

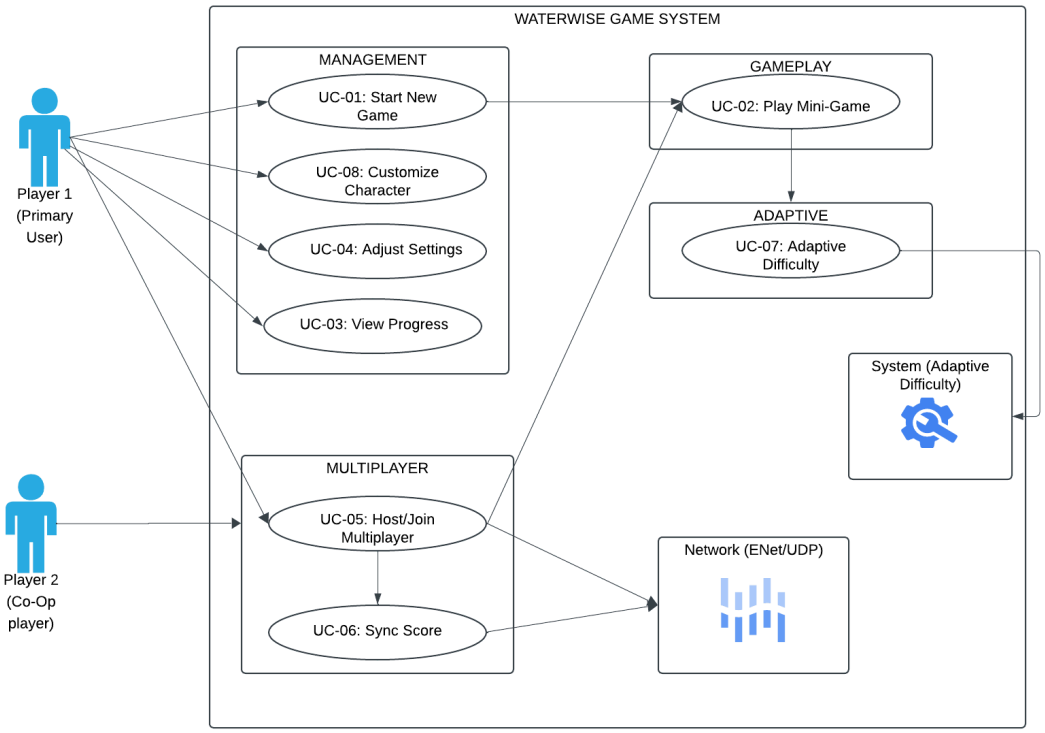


Figure 8. Primary Use Cases Key Use

Appendix 2

Tables

Sprint	Focus Area	Key Deliverables	Stakeholder Consultations
Sprint 1	Requirements & Architecture	- System Architecture & Layer Design - Circular Buffer Data Structure Definition (O(1))• Prioritized Product Backlog based on Survey/Interviews• Initial UI Mockups	- Educational Psychologists: Review of learning objectives and misconception targeting. - Game Designers: Consultation on core loop mechanics.
Sprint 2	Core Infrastructure	- Godot Project Structure & Modular Layers - "Game Manager" State Machine - Telemetry/Data Logging Pipeline (JSON Export) - Data Persistence Framework	- **Software Engineering Advisors:** Technical architecture review to validate bilingual support and modular design.
Sprint 3	Mini-Game Development (Batch 1)	- Physics Engine & Collision Logic - "SortWater" (Classification) Module	- **Domain Expert:** Verification of household water scenarios (e.g., informal plumbing)

		- "CatchTheRain" (Resource) Module	- **Educational Psychologist:** Validation that "SortWater" addresses the 53% safety misconception.
Sprint 4	Adaptive Difficulty Algorithm	- Adaptive Engine Implementation - Weighted Moving Average (WMA) & Consistency Penalty (CP) Logic - Proficiency Index (Phi) Calculation - Rule-Based Decision Tree	- Simulation Review: Validation of algorithm stability against erratic user behavior. - Expert Consultation: Review of pedagogically appropriate difficulty progression curves
Sprint 5	Mini-Game Development (Batch 2) & Localization	- Expansion of Content Library (7 additional games) - English-Filipino Translation Database - Hash-map Lookup System (Storage Optimization)	- Filipino Language Educator: Review of translations for linguistic accuracy and cultural appropriateness. - Expert Review: Validation of environmental narratives (droughts/floods).

		- Real-time Language Switching Logic	
Sprint 6	G-Counter CRDT & Multiplayer	- G-Counter CRDT Implementation - Custom UDP Networking Stack - Integer-Only Payload Optimization (Binary Protocol) - Network Partition/Reconnection Logic	- Distributed Systems Expert: Validation of CRDT correctness (Commutativity, Associativity, Idempotency) and bandwidth efficiency.
Sprint 7	Integration, Refinement & Documentation	- Full System Integration (Adaptive + Network + Render) - ISO 25010 Performance Profiling (FPS, Battery) - Post-Implementation Research Report - Final Thesis Documentation	- **Final Advisor Review:** Comprehensive audit of the complete system. - **Expert Panel:** Final evaluation of educational impact and technical soundness.

Table 3. Table Showing the sprint planning and activities

Software	Version	Purpose	Justification
Godot Engine	4.5	Game development framework	Open-source, cross-platform export (Windows, macOS, Linux, Android, iOS), GDScript integrated language, scene-based architecture, 2D optimization, educational licensing
Visual Studio Code	1.95	Code editor and IDE	GDScript syntax highlighting via extensions, integrated Git support, debugging capabilities, lightweight performance
Git	2.47	Version control system	Distributed architecture, branching for feature development, commit history for research documentation,collaboration support
GitHub	Webbased	Remote repository hosting	Cloud backup, version history, issue tracking; collaboration, thesis advisor access
Mermaid.js	11.4	Diagram generation	Code-based diagram specification, version control compatibility, flowcharts, sequence diagrams, ER diagrams
Canva	Webbased	Graphics editing	sprite creation, UI mockup generation, PNG export with transparency
Audacity	3.7	Audio editing	Open-source, sound effect creation, music editing; WAV/OGG export
Figma	Webbased	UI/UX design and prototyping	Collaborative design, component libraries, interactive mockups, stakeholder review

Table 4. Table showing the softwares selected for the development

Aspect	Godot Advantage	Educational Benefit	Research Support
Cost	Open-source, free	Sustainable for low-resource schools	Reduces implementation barriers
Deployment	Web, Mobile, Desktop	Maximum accessibility	Multi-platform effectiveness (Conmy, 2023)
Architecture	Scene-based, modular	Easy mini-game creation	Supports scalable development
Language	GDScript (Python-like)	Rapid development	Facilitates team collaboration
Cross-Platform	Single codebase	Consistent experience	Reduces development overhead

Table 5. Technology Stack Rationale with Research Support

Rule	Condition	Difficulty	Pedagogical Rationale	Actions	Research Support
1	$\Phi < 0.5$	EASY	Struggling/erratic performance	Increase time limits, enable hints, visual guidance	Maintains engagement, prevents frustration (Shohieb et al., 2025)
3	$0.5 \leq \Phi \leq 0.85$	MEDIUM	Optimal Flow State	Standard settings,	Flow theory foundation

				mild effects	(Kas Elias, 2024)
2	Φ > 0.85	HARD	Mastery with consistency	Faster speed, complex tasks, chaos effects	Challenges advanced learners (Silva et al., 2025)

Table 6. Decision Rules with Theoretical and Research Support

Metric Category	Specific Metrics	Evaluation Method	Success Criteria
Algorithmic Complexity	Time complexity; Space complexity	Big-O analysis of critical operations	Rolling window update O(1); CRDT merge O(1); Φ calculation O(1)
Convergence Guarantees	G-Counter eventual consistency	Mathematical proof of CRDT properties	Proof that commutativity, associativity, idempotency hold
Adaptation Accuracy	Difficulty classification correctness	Simulation with known performance patterns	>= 90% alignment with expert-validated appropriate difficulty
Responsiveness	Latency from input to output	Computational timing analysis	< 16ms processing latency (1 frame at 60 FPS) to ensure zero perceptible lag

Network Efficiency	Packet size; Sync frequency Synchronization frequency	Bandwidth analysis	< 1 KB per sync packet; Event-driven sync (only on score change)
Frame Rate Stability	FPS during adaptive calculations	Performance profiling (Godot Monitor)	≥60 FPS maintained during Φ calculation
Energy Efficiency	Battery Discharge Rate Power	Profiling (mAh)	< 10 mAh per 5-minute session (approx. < 2% drain on standard 4000mAh battery)
Memory Efficiency	RAM Usage	Memory Profiler	< 200 MB (approx. 10% of 2GB RAM budget to prevent OS kills)

Table 12. Performance Evaluation Metr

Appendix 3

Letter

Republic of the Philippines
CAVITE STATE UNIVERSITY
Don Severino de las Alas Campus
Indang, Cavite
(046) 483-9250
www.cvsu.edu.ph

BAGONG PILIPINAS

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

DR. DAISY A. PELLOS
ICA, Principal
Indang Christian Academy

November 13, 2025

Dear Dr. Pellos,

Greetings! We hope you are doing well.

We are fourth-year Bachelor of Science in Computer Science students from Cavite State University – Indang Campus, currently working on a research project titled "WaterWise: A Rule-Based Adaptive Difficulty Game for Water Reuse Education Among Filipino Elementary Students." As part of our study, we respectfully request permission to conduct a research survey at Indang Christian Academy under your supervision.

The survey aims to gather feedback from Grade 3–6 students to help us assess and enhance the game's content, adaptive difficulty system, and developmental suitability. All participation will be voluntary, and every response will be handled with strict confidentiality, used solely for academic purposes.

We assure you that we will follow all university and school guidelines and will coordinate with the school administration to ensure the activity does not disrupt regular classes or school operations.

We sincerely hope for your approval and support in carrying out this research initiative. Your assistance will greatly contribute to the success of our study and to the development of educational tools that promote water conservation among young learners.

Thank you very much for your time and kind consideration.
Sincerely,

RALPH LAURENCE J. CRUTO
Student

RAEVEN M. ESPINELI
Student

ELRONI F. QUINONES
Student

Noted by:

JULIE ANN C. LONTOC
Adviser

EZRA MARIE F. RAMOS
Technical Critic

Recommending Approval:

WILLIE C. BUCLATIN
Dean, College of Engineering and Information Technology
Cavite State University - Don Severino Delas Alas Campus

Approved By:

DR. DAISY A. PELLOS
Principal, Indang Christian Academy